

RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation

Anonymous Authors¹

Abstract

Retrieval-augmented language models often struggle with knowledge-intensive tasks due to inefficient retrieval, unstructured knowledge integration, and single-pass architectures. We present Retrieval-And-Structuring (RAS), a novel framework that dynamically constructs and reasons over query-specific knowledge graphs through iterative retrieval and structuring. RAS introduces four key technical innovations: (1) a theme-scoped retrieval mechanism that efficiently narrows the search space while maintaining retrieval quality, (2) an action planning module that determines knowledge needs and generates focused sub-queries, (3) a dynamic knowledge structuring approach that converts retrieved text into an evolving knowledge graph, and (4) a graph-augmented answering component that leverages the accumulated structured information. Our framework achieves state-of-the-art performance, surpassing leading baselines by 6.4% with open-source language models and 7.0% with proprietary models on seven knowledge-intensive generation datasets across all evaluation metrics. Detailed ablation studies verify the contribution of each technical component to the overall system performance.

1. Introduction

Complex reasoning tasks like scientific analysis or multi-hop question answering demand both comprehensive knowledge and structured logical thinking. While large language models (LLMs) have achieved remarkable performance across natural language processing tasks (Devlin et al., 2018; Brown et al., 2020; Chowdhery et al., 2023), they often struggle with knowledge-intensive reasoning due to the absence of precise, logically structured information (Rae et al., 2021; Ling et al., 2024). This limitation has prompted growing

research into augmenting LLMs with structured knowledge to enhance their reasoning capabilities (Wang et al., 2021).

Retrieval-augmented generation (RAG) approaches have emerged as a promising direction by providing LLMs with additional context from retrieved passages (Guu et al., 2020; Lewis et al., 2020; Izacard & Grave, 2021; He et al., 2024). However, these methods frequently face hallucination challenges (Maynez et al., 2020; Zhang et al., 2023b), where generated content deviates from retrieved information. This issue stems primarily from the unstructured nature of retrieved passages, which lack explicit logical connections needed for precise reasoning. Recent efforts have explored integrating knowledge graphs (KGs) with LLMs (Sun et al., 2019; Yu et al., 2022; He et al., 2024; Edge et al., 2024), as KGs offer compact, structured representations of entities and relationships that enable more accurate and interpretable reasoning (Hogan et al., 2021; Jiang et al., 2024).

We present Retrieval-And-Structuring (RAS), a novel framework that dynamically constructs and reasons over query-specific knowledge graphs through iterative retrieval and structuring. RAS integrates theme-aware retrieval, dynamic KG construction, and self-refinement (Ji et al., 2023; Asai et al., 2023; Lyu et al., 2024b) to comprehensively prepare knowledge for challenging questions. The framework comprises four key components: (1) an action planner that strategically determines knowledge needs and generates targeted sub-queries, (2) a theme-scoped retrieval module that efficiently combines thematic and semantic relevance, (3) a knowledge structuring module that builds and expands query-specific KGs, and (4) a knowledge-augmented answerer that leverages the gathered structured information.

RAS addresses several critical limitations of existing approaches. First, unlike traditional RAG methods that rely solely on semantic similarity (Lewis et al., 2020; Guu et al., 2020), RAS employs theme-aware retrieval that significantly reduces the search space while preserving retrieval quality. Second, in contrast to static KG approaches (He et al., 2024; Edge et al., 2024), RAS dynamically constructs and evolves KGs based on the specific reasoning requirements of each query. Third, through its iterative planning mechanism, RAS actively identifies knowledge gaps and initiates focused retrievals, moving beyond the single-pass limitations of existing methods (Izacard & Grave, 2021).

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

On seven knowledge-intensive tasks including open-domain QA, closed-set QA, and long-form generation, RAS outperforms state-of-the-art baselines by 6.4% with open-source LLMs and 7.0% with proprietary models. The main contributions of this work:

- We introduce RAS, a novel framework that dynamically constructs query-specific knowledge graphs through iterative retrieval and structuring for knowledge-intensive language generation.
- We propose theme-scoped retrieval that combines thematic classification with dense retrieval to efficiently narrow the search space while maintaining retrieval quality.
- We develop a dynamic knowledge structuring approach that iteratively builds and enriches knowledge graphs based on reasoning needs.
- We validate RAS’s effectiveness through comprehensive experiments and ablation studies across diverse knowledge-intensive tasks.

2. Related Work

Retrieval-Augmented Generation. Retrieval-Augmented Generation (RAG) (Lewis et al., 2020; Guu et al., 2020) enhances performance on knowledge-intensive tasks by incorporating retrieved passages into input of large language models (LLMs), improving factual accuracy and grounding. Traditional approaches retrieved a fixed number of passages once before generation (Shao et al., 2023; Es et al., 2024; Lyu et al., 2024a) or focused on adaptive retrieval (Jiang et al., 2023), where models dynamically query external knowledge when confidence is low, or analyze on the retrieval (Kim et al., 2024b). However, these methods often fail to perform complex or multi-hop reasoning process due to the linear and simple designs, where several recent methods have emerged to improve retrieval quality. Iterative retrieval-generation approaches (Shao et al., 2023; Guan et al., 2024) leverage historical context to enhance subsequent retrievals. Other work introduces targeted retrieval strategies through subqueries (Khatab et al., 2023) or follow-up questions (Yao et al., 2023; Press et al., 2022), progressively enriching the context for more factually grounded generation. SelfRAG (Asai et al., 2023) introduced self-reflective RAG, which allows LLMs to retrieve content on demand and critically evaluate retrievals, demonstrating both the importance of retrieval quality and the capacity of LLMs to seek self-directed information. RPG (Lyu et al., 2024b) further refined this approach by extracting fine-grained paragraphs from retrieved content to improve query relevance. However, existing methods face key limitations.

They often either include irrelevant content in the retrieved context or risk discarding essential information through refinement. Additionally, dense retrieval approaches can be computationally expensive when deployed on large corpora, hampering real-time user interaction. To address these challenges, we propose converting retrieved text into knowledge-intensive graphs and maintaining an iteratively enriched, query-specific knowledge graph for knowledge serving. We also introduce theme-scoped retrieval for more focused and efficient context retrieval.

Graph as Context for LLMs. The integration of graphs into modern language models represents a promising research direction (Yasunaga et al., 2021; 2022; Yu, 2022; Ju et al., 2022; Zhu et al., 2024; Gutiérrez et al., 2024). Recent approaches leverage graph-based knowledge representation to enhance LLM performance. For instance, GraphToken (Perozzi et al., 2024) shows that LLM can understand a graph encoded as a token, after a simple p-tuning (Liu et al., 2021) while keeping the LLM’s parameters frozen. Based on the findings, G-Retriever (He et al., 2024) constructs subgraphs by retrieving relevant entities and relations from a global KG, using these as structured knowledge input for LLMs. GraphRAG (Edge et al., 2024) employs an alternative strategy, extracting a comprehensive global KG from the corpus, identifying meaningful clusters through graph community detection, and generating LLM-based summaries of these communities to create a refined retrieval corpus. These methods demonstrate how graphs, with their knowledge-intensive modality and inherent structural properties, can significantly improve LLMs’ contextual understanding. However, current approaches treat graphs as static data structures without refinement based on LLM requirements. Moreover, building global KGs for large-scale corpora remains prohibitively expensive and often impractical – Wikipedia alone contains over 30 million documents per dump. The key challenge lies in developing methods to dynamically construct and utilize graphs on-the-fly for knowledge-intensive generation. To address these limitations, we propose a retrieval-and-structuring paradigm that dynamically builds and maintains query-specific KGs on-the-fly. Our approach iteratively enriches these graphs through LLM’s self-evaluation of information completeness and targeted retrieval via focused subqueries.

3. RAS Framework

Knowledge-intensive language generation requires not just retrieving relevant information, but also structuring and reasoning over it effectively. We propose Retrieval-And-Structuring (RAS), a framework shown in Figure 1. In the following subsections, we detail the technical mechanisms underlying each stage. To facilitate understanding, we showcase a running example of RAS in Figure 26.

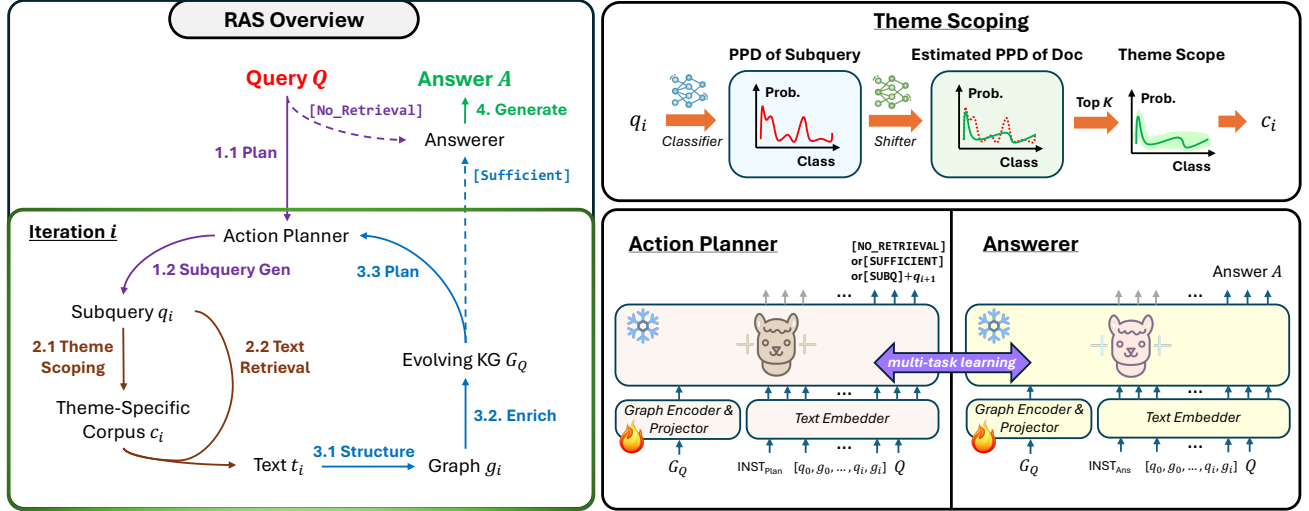


Figure 1. Overview of the Retrieval-And-Structuring (RAS) framework. RAS operates through four integrated stages: (1) **Action Planning** strategically determines knowledge retrieval needs and generates focused sub-queries; (2) **Theme-Scoped Retrieval** combines thematic and semantic matching to efficiently extract relevant context from large corpora; (3) **Knowledge Structuring and Enrichment** transforms retrieved text into an evolving knowledge graph that expands based on reasoning needs; and (4) **Knowledge-Augmented Generation** leverages the accumulated structured knowledge through a GraphLLM to produce the final output. The framework employs parameter-efficient training, fine-tuning only the graph encoder, projector components, and optionally with LoRA. The Action Planner and Answerer are two roles played by a single model, trained through multi-task learning.

3.1. Action Planning

The Action Planning stage is the first critical step in the RAS framework, guiding the retrieval and reasoning process based on the input query and the current knowledge state. It involves two key components: the Action Planner $\mathcal{M}_{plan}$, a language model that assesses the sufficiency of the current knowledge state and generates sub-queries for further retrieval if necessary, and a decision step that determines the next action based on the planner’s output.

(For initial iteration) Formally, given an input query Q , the Action Planner $\mathcal{M}_{plan}$ generates a plan p_0 according to:

$$p_0 \leftarrow \mathcal{M}_{plan}(\text{None}; \text{None}; Q) \quad (1)$$

where p_0 can take one of two forms:

- **[NO_RETRIEVAL]:** If $\mathcal{M}_{plan}$ determines that Q can be answered directly without requiring any additional knowledge, the planning process terminates, and the framework proceeds directly to the final Answer Generation stage (§3.4).
- **[SUBQ] $q_0 = Q$:** If $\mathcal{M}_{plan}$ assesses that the query cannot be satisfactorily answered with its own knowledge, we start the iteration with the main query Q as the initial subquery, and move to the next stage (§3.2).

(For iteration > 0) At each iteration i , the Action Planner reassesses the cumulative retrieved knowledge G_i and

the chain of previous sub-queries $(q_0, g_0), \dots, (q_i, g_i)$ in the context of the main query Q :

$$p_{i+1} \leftarrow \mathcal{M}_{plan}(f_{gnn}(G_i), (q_0, g_0), \dots, (q_i, g_i), Q) \quad (2)$$

where f_{gnn} is a graph neural network for encoding the evolving KG G_i ; q_k is the subquery at iteration k , and g_k is the extracted graph information (triple list) from the retrieved context. The base model we use here is GraphLLM, an architecture derived from previous work (Perozzi et al., 2024; He et al., 2024). We discuss more details in §3.3.

The planner’s output p_{i+1} at each iteration can be either:

- **[SUFFICIENT]:** The accumulated knowledge G_i is deemed sufficient to comprehensively address the main query Q . The iterative retrieval process terminates, and the framework proceeds to the Knowledge-Augmented Generation stage (§3.4).
- **[SUBQ] q_{i+1} :** The planner generates a new subquery q_{i+1} to guide the retrieval of additional relevant knowledge. The subquery is designed to fill specific gaps in the current knowledge state with respect to answering Q . The framework continues the iterative process and proceeds to the Theme-Scoped Retrieval stage (§3.2).

The Action Planner serves as a key driver of the RAS framework’s iterative retrieval and refinement process. By dynamically assessing the adequacy of the retrieved knowledge

and generating targeted sub-queries, it enables the efficient acquisition of query-relevant information.

3.2. Theme-Scoped Retrieval

This stage efficiently retrieves thematically and semantically relevant context for subquery q_i through a two-step process: (1) theme scoping to identify a narrowed sub-corpus c_i close to q_i 's thematic distribution, and (2) dense retrieval to find the most relevant passages t_i within c_i . This thematic pre-filtering before semantic matching aims to improve both retrieval efficiency and precision.

3.2.1. THEME SCOPING

The Theme Scoping mechanism transforms sub-queries into thematic probability distributions and retrieves relevant document subsets through three key operations:

$$\text{PPD}(q_i) \leftarrow f_{cls}(q_i) \quad (3)$$

$$\text{PPD}(d_i) \leftarrow f_{shift}(\text{PPD}(q_i)) \quad (4)$$

$$c_i \leftarrow \text{ThemeSim}(\text{PPD}(d_i), C_{PPD}, K) \quad (5)$$

First, a theme classifier f_{cls} maps the subquery q_i to a posteriori probability distribution (PPD) over thematic classes (Eq. 3). The classifier is trained on DBpedia-298 (Lehmann et al., 2015)¹, a multilabel wiki text classification dataset, enabling fine-grained thematic distinctions.

To address the distribution shift between queries and documents, an MLP-based shifter f_{shift} transforms the query theme distribution $\text{PPD}(q_i)$ into an expected document theme distribution $\text{PPD}(d_i)$ (Eq. 4). This shifter is trained on query-document pairs in our constructed dataset HotpotQA-SUBQ (see Appendix B), learning to capture the relationship between query and document thematic patterns.

Finally, using the shifted distribution $\text{PPD}(d_i)$, we retrieve a theme-specific sub-corpus c_i from the theme-indexed corpus C_{PPD} (Eq. 5). The $\text{ThemeSim}(\cdot)$ function selects the top- K documents based on L2 similarity between $\text{PPD}(d_i)$ and the pre-computed theme distributions in C_{PPD} . This focused retrieval process effectively reduces the search space while maintaining thematic relevance for subsequent semantic matching operations.

3.2.2. DENSE RETRIEVAL

Within the theme-specific sub-corpus c_i , a dense retriever f_{den} is employed to identify the most semantically relevant passages t_i for the subquery q_i :

$$t_i \leftarrow \text{DenseSim}(f_{den}(q_i), f_{den}(c_i), k) \quad (6)$$

¹Document label space of DBpedia-298: <https://mappings.dbpedia.org/server/ontology/classes/>

where $\text{DenseSim}(\cdot)$ selects the top- k passages based on the cosine similarity between the query embedding $f_{den}(q_i)$ and the passage embeddings $f_{den}(c_i)$ precomputed in the dense index.

3.3. Knowledge Structuring and Enrichment

This stage is designed to transform the retrieved text passages t_i into a structured graph representation g_i and iteratively expand the query-specific knowledge graph G_Q .

3.3.1. TEXT-TO-TRIPLE CONVERSION

To capture the essential facts from the retrieved passages t_i , a Flan-T5-Large (Chung et al., 2024) based text-to-triples model f_{t2t} is employed. We train the model on the full set of WikiOFGraph dataset (Kim et al., 2024a), an LLM-curated high-quality text-triples dataset, enabling it to generate quality and information-rich triples in the format:

$$g_i \leftarrow f_{t2t}(t_i) = [(s_0, r_0, o_0), \dots, (s_{|g_i|}, r_{|g_i|}, o_{|g_i|})] \quad (7)$$

where each triple (s_j, r_j, o_j) represents a subject-predicate-object fact extracted from the text. This structured representation allows for efficient downstream reasoning and simple external knowledge graph integration.

3.3.2. ITERATIVE KNOWLEDGE ENRICHMENT

The extracted triples g_i are then converted into a graph structure $g'_i = (V_i, E_i)$, where V_i and E_i denote the sets of nodes and edges, respectively. Each node $v \in V_i$ corresponds to a unique subject or object entity in g_i , while each edge $e \in E_i$ represents a predicate connecting two entities. To enrich the graph with semantic information, the attributes of nodes and edges are obtained through a text encoder f_{enc} (Sentence-BERT (Reimers, 2019) in our case):

$$v.attr \leftarrow f_{enc}(v), \forall v \in V_i \quad e.attr \leftarrow f_{enc}(e), \forall e \in E_i \quad (8)$$

These semantic embeddings enable the model to capture the nuanced relationships between entities and facilitate reasoning over the knowledge graph.

To progressively expand the knowledge base in response to the evolving sub-queries, the structured graph g'_i at each iteration i is merged into an evolving knowledge graph $G_Q = (V_Q, E_Q)$ specific to the main query Q :

$$G_Q \leftarrow G_Q \cup g'_i \quad (9)$$

After enriching G_Q with the new knowledge, the action planner (§3.1) reassesses the current knowledge to determine the next step. Based on G_Q and the chain of previous subqueries and their associated graph information, the planner decides whether to generate another focused subquery for additional retrieval (§3.2) or to proceed with answering (§3.4) if the accumulated knowledge is deemed sufficient.

3.4. Knowledge-Augmented Generation

The final stage generates an answer A to the main query Q using the Answerer model $\mathcal{M}_{ans}$ conditioned on knowledge graph G_Q and subquery chain $(q_0, g_0), \dots, (q_i, g_i)$.

If no retrieval is needed ($p_0 = [\text{NO_RETRIEVAL}]$), the answer is generated directly:

$$A \leftarrow \mathcal{M}_{ans}(\text{None}; \text{None}; Q) \quad (10)$$

Otherwise, after iterative knowledge enrichment concludes with $[\text{SUFFICIENT}]$ plan, the answer is generated using encoded KG $f_{gmn}(G_Q)$ and subquery chain:

$$A \leftarrow \mathcal{M}_{ans}(f_{gmn}(G_Q); (q_0, g_0), \dots, (q_i, g_i); Q) \quad (11)$$

$\mathcal{M}_{ans}$ attends to relevant knowledge in G_Q and subquery chain to generate accurate, coherent answers.

Note: In this study, $\mathcal{M}_{ans}$ and $\mathcal{M}_{plan}$ are two roles played by the same model with different instructions (see §C.3).

4. Experiments

Training Data & Setting. We develop HotpotQA-SUBQ, a dataset derived from HotpotQA (Yang et al., 2018), to train our model’s action planning and answering capabilities. Our dataset creation begins with document filtering: using Claude 3.5 Sonnet (Anthropic, 2024), we identify and retain only the supporting documents necessary for answering the main query, removing irrelevant content. For each supporting document d_j , we then iteratively generate a subquery q_j , considering the main query, previous subqueries, and supporting documents. During iteration j , when more supporting documents remain, we create training samples with input $\{q_0, g_0, \dots, q_j, g_j, Q\}$ and output label “ $[\text{SUBQ}] q_{j+1}$ ”, where g_k represents triples extracted from document d_k , and Q is the main query. For the final supporting document, we label the sample as “ $[\text{SUFFICIENT}]$ ”. To identify queries that can be answered directly, we test our base LLM (LLaMA-2-7B) on HotpotQA’s main queries without context. For correctly answered queries, we create training samples with the main query Q as input and “ $[\text{NO_RETRIEVAL}]$ ” as the output label. Additionally, to ensure fair comparison with existing approaches, we incorporate the subset of Arc-Easy (2,147 samples) and ASQA (3,897 samples) from SelfRAG’s training data, resulting in 208k training samples in total. We place detailed training data processing, dataset statistics, and data samples in Appendix B. We use DBPedia-298, HotpotQA-SUBQ, and WikiOfGraph to train our theme classifier f_{cls} , distribution shifter f_{shift} , and text-to-triples model f_{t2t} , respectively, as detailed in Appendix C. We present our hyperparameter study of each component in Appendix G.

Knowledge Sources. Due to the large sizes of wiki corpora, we employ faiss (Douze et al., 2024) for efficient vector

searching across both theme and dense indices. Following the SelfRAG (Asai et al., 2023), we utilize the Wikipedia 2018 (Izacard et al., 2023) by default, while specifically using the Wikipedia 2020 for PopQA to access more recent information. To optimize retrieval efficiency, we partition the index into five segments. Within each segment, we first perform theme scoping to reduce the candidate document pool K to 250,000 documents by default. However, we skip theme scoping for PopQA since our classifier f_{cls} , trained on DBPedia-298 data based on old Wiki dump, performs poorly on newer Wiki 2020 dump, producing near-zero vectors for roughly one-third of recent content.

Test Datasets & Metrics & Compared Baselines. We conduct comprehensive evaluations on diverse knowledge-intensive tasks following previous studies (Asai et al., 2023; Lyu et al., 2024b). The evaluation encompasses three categories of datasets: (1) open-domain short-form generation datasets: TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), and 2WikiMultihopQA (Ho et al., 2020); (2) closed-set task datasets: PubHealth (Zhang et al., 2023a) and ARC-Challenge (Clark et al., 2018); and (3) long-form generation datasets: ALCE-ASQA (Gao et al., 2023; Stelmakh et al., 2022) and ELI5 (Fan et al., 2019). For evaluation metrics, we maintain consistency with prior work (Asai et al., 2023; Mallen et al., 2022; Lyu et al., 2024b), employing “golden match” accuracy for PopQA and TriviaQA, token-level F1 score for 2WikiMultihopQA, accuracy for PubHealth and ARC-Challenge, and ROUGE-LSum alongside MAUVE score (Pillutla et al., 2021) for ASQA and ELI5. Our comparative analysis includes three baseline categories: models without retrieval augmentation, incorporating Claude 3.5 Sonnet as a state-of-the-art closed-source baseline; models with single retrieval over top-5 documents, including Claude 3.5 Sonnet, and SuRe (Kim et al., 2024b), a leading retrieve-and-summarize framework; and models with self-reflective retrieval, including leading approaches Self-RAG (Asai et al., 2023) and RPG (Lyu et al., 2024b). We place more details of metrics/datasets in Appendix D.

Inference Setting. We evaluate RAS using both our trained open-source model (RAS_{7B/8B} with LLaMA-2-7B/LLaMA-3-8B and Graph Transformer (Shi et al., 2020) encoder) and the closed-source Claude-3.5-Sonnet model under varied inference settings. For RAS_{7B/8B}, we maintain consistency with previous work (Asai et al., 2023; Lyu et al., 2024b) by employing zero-shot inference across all datasets. For closed-source models (e.g., RAS_{Sonnet-3.5}), we utilize few-shot inference with two exemplars for ASQA and ELI5, while maintaining zero-shot for the other datasets.

During inference, if the initial planner output is not $[\text{NO_RETRIEVAL}]$, we utilize the main query Q as the initial subquery q_0 . For PopQA and TriviaQA evaluation, we follow established settings (Asai et al., 2023; Luo et al.,

Model	Short-form			Closed-set		Long-form Generation			
	TQA (acc)	2WQA (F1)	PopQA (acc)	Pub (acc)	ARC (acc)	ASQA		ELI5	
						(rouge)	(mauve)	(rouge)	(mauve)
<i>Baselines without retrieval</i>									
Llama2 _{7B}	30.5	18.9	14.7	34.2	21.8	15.3	19.0	18.3	32.4
Alpaca _{7B}	54.5	–	23.6	49.8	45.0	29.4	61.7	–	–
Llama2 _{13B}	38.5	20.2	14.7	29.4	29.4	12.4	16.0	18.2	41.4
Alpaca _{13B}	61.3	–	24.4	55.5	54.9	32.0	70.6	–	–
ChatGPT _{~175B}	74.3	24.8	29.3	70.1	75.3	36.2	68.8	22.8	32.6
Sonnet-3.5 _{~175B}	78.4	40.0	30.2	83.7	88.5	37.0	39.1	21.8	26.5
<i>Baselines with retrieval (#docs=5 by default)</i>									
Llama2 _{7B}	42.5	21.0	38.2	30.0	48.0	22.1	32.0	18.6	35.3
Alpaca _{7B}	64.1	–	46.7	40.2	48.0	33.3	57.9	–	–
Llama2 _{13B}	47.0	31.2	45.7	30.2	26.0	20.5	24.7	18.6	42.3
Alpaca _{13B}	66.9	–	46.1	51.1	57.6	36.7	56.6	–	–
RECOMP _{7B} (Xu et al., 2023)	–	32.4	–	–	–	36.5	76.0	–	–
SuRe _{7B} (Kim et al., 2024b)	–	20.6	–	–	–	35.8	76.2	–	–
SuRe _{GPT}	72.3	38.1	53.6	57.2	79.6	36.0	74.2	–	–
ChatGPT	65.7	32.8	50.8	54.7	75.3	39.9	79.7	20.6	57.2
Sonnet-3.5 _{#docs=1}	69.1	41.9	51.5	49.1	88.6	–	–	–	–
Sonnet-3.5 _{#docs=5}	72.5	53.7	57.3	53.9	87.1	38.8	61.6	20.2	32.3
SuRe _{Sonnet-3.5}	76.8	37.6	41.2	62.8	91.6	30.2	69.9	15.4	27.2
<i>Baselines with self-reflective retrieval</i>									
SELF-RAG _{7B} (Asai et al., 2023)	66.4	25.1	54.9	72.4	67.3	35.7	74.3	17.9	35.6
SELF-RAG _{13B}	69.3	–	55.8	74.5	73.1	37.0	71.6	–	–
RPG _{7B} (Lyu et al., 2024b)	–	33.6	56.0	73.4	–	37.6	84.4	19.1	46.4
<i>Retrieval-And-Structuring (Ours)</i>									
$\mathcal{M}_{Plan}$	$\mathcal{M}_{Ans}$								
RAS _{Sonnet-3.5}	RAS _{Sonnet-3.5}	77.4	57.7	62.3	71.4	93.8	39.1	70.5	23.3
RAS _{Sonnet-3.5}	RAS _{7B}	73.8	43.4	61.3	75.5	62.4	37.2	95.2	19.7
RAS _{7B}	RAS _{Sonnet-3.5}	78.7	54.7	58.8	70.8	91.2	39.1	70.5	23.3
RAS _{7B}	RAS _{7B}	72.6	42.1	58.3	74.8	67.7	37.2	95.2	19.7
RAS _{8B}	RAS _{8B}	73.8	44.2	57.7	77.5	71.5	37.6	96.2	20.1

Table 1. Performance Comparison. We highlight the **top-2 closed-source models** and **top-2 open-source 7B models**, and underline the best closed-/open-source (7B) model for each dataset. **Note:** Either RAS_{7B} or RAS_{8B} is a single model trained with action planner’s and answerer’s instructions via multi-task learning. No ensemble or multi-agent approach is used for any experiments.

2023), incorporating top-five web search engine results as initial retrieved context t_0 . For ASQA and ELI5, we maintain methodological consistency with prior work (Lyu et al., 2024b; Asai et al., 2023; Gao et al., 2023), utilizing their predetermined five-document context. Under these conditions, we omit plan generation and text retrieval phases, implementing static inference (Asai et al., 2023) with five fixed iterations. For remaining datasets, we establish a maximum iteration count of five. Following previous studies, we employ Contriever-MS MARCO (Izacard et al., 2021) as the primary dense retriever, with BM25 (Robertson et al., 2009) serving as the retrieval mechanism for 2WikiMultihopQA. Across all retrieval processes, we maintain a consistent top- k document selection of five. We present more details of inference settings in Appendix E.

4.1. Main Results

Our performance evaluation, presented in Table 1, demonstrates that the LLaMA-2-7B/LLaMA-3-8B model fine-tuned with RAS outperforms existing open-source solutions, including SelfRAG (Asai et al., 2023) and RPG (Lyu et al., 2024b). Notably, compared to the previous SOTA models, RAS_{7B} shows a 9.7% improvement in short-form question-answering and a 7.9% gain in long-form generation tasks. Additionally, when applied to closed-source Sonnet-3.5 model, RAS consistently achieves superior results compared to single retrieval RAG approaches, including retrieve-and-summarize approach SuRe (Kim et al., 2024b).

We find that sometimes (e.g., on TriviaQA and PubHealth) single-hop retrieval could not boost LLM’s performance,

	TQA (acc)	2WQA (F1)	Pub (acc)	ASQA (rg)	ASQA (mv)
RAS _{7B}	72.6	42.1	74.8	37.2	95.2
<i>Training Phase</i>					
No GraphToken	70.2	38.4	66.4	33.1	85.0
No LoRA	71.5	37.8	54.8	32.8	84.8
No Text-to-Triple	70.4	38.2	71.4	36.2	73.8
No Mutli-Task	68.6	39.2	65.5	36.7	88.9
<i>Inference Phase</i>					
No Retrieval	56.9	27.4	69.0	31.3	70.6
No GraphToken	68.8	38.7	67.3	36.5	93.6
No Planning	66.7	37.8	71.5	–	–
No ThemeScope	72.0	–	73.9	–	–

Table 2. Ablations in Training and Inference (with RAS_{7B}).

Training: “No GraphToken” removes the graph encoder, using only LoRA-based LLM fine-tuning. “No LoRA” uses graph token optimization without low-rank adaptation. “No Text-to-Triple” keeps the original retrieved text instead of constructing KGs. “No Multi-Task” trains separate action planner and answerer rather than training a single model. **Inference:** “No Retrieval” tests direct query answering without any context. “No GraphToken” removes graph structure during inference, using only textual context. “No Planning” uses single retrieval. “No ThemeScope” bypasses theme-aware retrieval, using only dense retrieval.

and even makes it worse, which demonstrates the necessity of on-demand retrieval, aligning with previous findings.

Our “role-swapping” study, where Sonnet-3.5 and RAS_{7B} alternately serve as action planner and answerer, reveals that performance is primarily bounded by the answerer’s inherent capabilities. For example, when Sonnet-3.5 acts as the planner with RAS_{7B} as the answerer, the system achieves 62.4% accuracy on the ARC-Challenge dataset. However, when Sonnet-3.5 serves as both planner and answerer, accuracy increases to 93.8%. Notably, despite having 60x fewer parameters, our trained RAS_{7B} performs comparably to Sonnet-3.5 as a planner, and even outperforms it sometimes.

Also, for long-form generation task, our trained model achieved notably high MAUVE scores, which can be majorly attributed to our “text-to-triple” generation step, as demonstrated by our later ablation study in Table 2.

4.2. Ablations & More Analysis

We study the effect of individual component in Table 2.

Effect of Iterative Planning and Retrieval. Comparing the base model with “No Planning” variant shows that iterative planning provides consistent improvements across all metrics (e.g., +8.8% on TQA, +9.0% on 2WQA). This demonstrates the importance of dynamically determining retrieval needs and generating focused sub-queries. Without

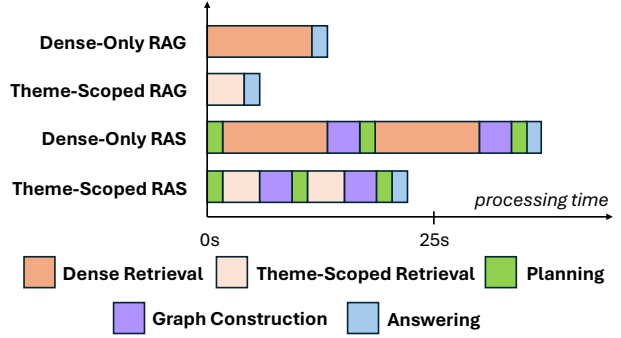


Figure 2. Efficiency study of RAS conducted on our machine. For RAS, we set #iteration=2 and use closed-source LLM for illustration. The result is averaged over 100 TriviaQA samples tested. $K=250,000$ and $k=5$ are set in this case. The Wiki 2018 dump ($\sim 32M$ documents) is used and loaded on CPU for this analysis.

planning, the model relies on single-pass retrieval, which may miss crucial information needed for complex reasoning. Also, when turning off retrieval, the performance degradation is more obvious, due to the knowledge-intensive nature of those datasets.

Effect of Graph Construction and Encoding. The impact of structured knowledge representation is evident from two ablations. First, “No Text-to-Triple” degrades performance significantly on all metrics (e.g., -9.0% on 2WQA, -22.2% MAUVE on ASQA), showing the value of essential information extraction via converting retrieved text into structured triples. Second, removing the graph encoder (“No GraphToken”) during training or inference consistently hurts performance across datasets, with particularly large drops on PubHealth (-11.2% and -10.0% respectively). This suggests that the graph structure encoding helps the model better leverage the knowledge relationships.

Effect of LoRA and Multi-task Learning. Our experiments reveal that parameter-efficient training strategies significantly impact model performance. Using only graph token optimization without LoRA leads to substantial degradation (-11.8% on average). A similar observation can be made for “No Multi-Task” (e.g., 12.3% accuracy degradation on PubHealth), indicating the significance of jointly training the model on both action planning and answer generation tasks rather than optimizing for each task separately, supporting findings from prior work (Lyu et al., 2024b). The complementary effects suggest that while graph-based knowledge representation is valuable, it needs to be combined with careful parameter tuning and multi-task learning to achieve optimal performance.

Effect of Theme Scoping. The theme-scoped retrieval shows mixed results – while removing it (“No ThemeScoping”) has minimal performance drops on TQA and PubHealth, the efficiency benefits are substantial as shown in

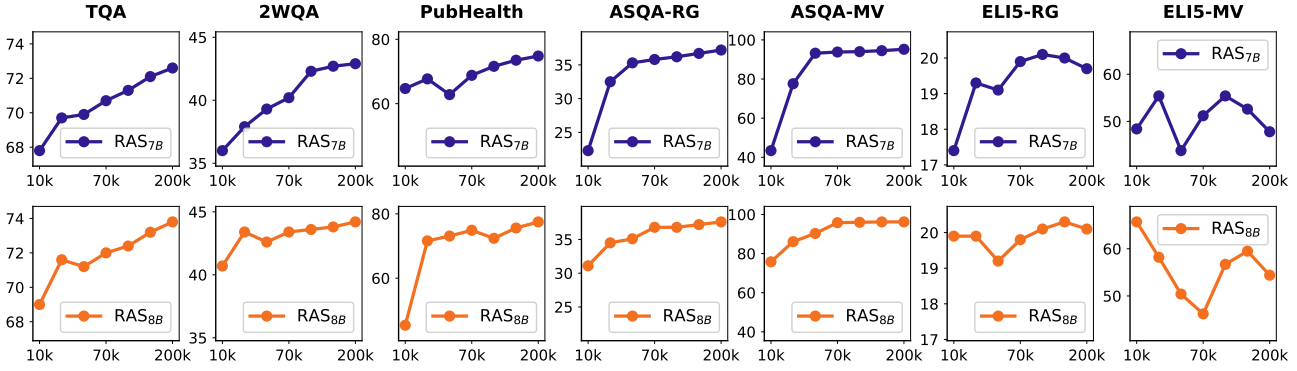


Figure 3. **Impact of Training Data Volume on Model Performance.** Results for RAS_{7B} (top) and RAS_{8B} (bottom) illustrate how performance scales with increasing training data.

Figure 2. This suggests theme scoping successfully narrows the search space without compromising performance.

Efficiency Analysis. Figure 2 compares the computational costs of different retrieval strategies. Theme-scoped RAG reduces processing time by approximately 60% compared to dense-only RAG by first filtering the corpus through lightweight theme classification (with 298 class dimensions). This combination of comparable performance with significantly reduced computational cost demonstrates the effectiveness of our theme-scoped retrieval approach.

Impact of Training Data Volume. Figure 3 demonstrates how training dataset size influences model performance across different tasks. Considering the computational efficiency, we sampled 2,000 instances each from TQA and 2WQA for evaluation, while maintaining the original sizes of other datasets. The results indicate that model performance generally improves with increased training data volume. Notably, our model achieves competitive performance even with limited training data - using only 5% (10K instances) of our full dataset, it surpasses previous state-of-the-art models on TQA, 2WQA, and ELI5. These results suggest both the robustness of our architectural design and the effectiveness of our data curation methodology. However, we observed an exception with the ELI5 dataset, where performance were inconsistent. This irregularity can be attributed to the inclusion of ASQA training data in our training set, following established setting from previous research (Asai et al., 2023; Lyu et al., 2024b). Among our test datasets, ASQA and ELI5 are unique in requiring long-form response generation. The periodic decline in ELI5 performance suggests that the model’s response generation began to align more closely with ASQA’s training data distribution, potentially at the expense of ELI5’s distinct characteristics.

Impact of Graph Information Abundance. Figure 4 demonstrates a strong positive correlation between graph information abundance and model performance across all

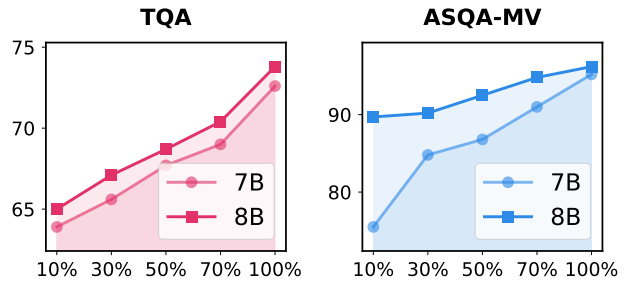


Figure 4. **Impact of Graph Information Abundance on Model Performance.** For each sample, we randomly shuffle its associated triples five times and take different ratios (10%, 30%, 50%, 70%, 100%) of the shuffled data. The performance scores shown are averaged across these five shuffling runs.

tasks. Both RAS_{7B} and RAS_{8B} show consistent improvements as graph information increases, demonstrating RAS’s strong capability of effective utilizing available graph information for knowledge-intensive generation.

5. Conclusion

We presented RAS (Retrieval-And-Structuring), a novel framework that enhances knowledge-intensive language generation through iterative retrieval and knowledge structuring. Our approach introduces key innovations including theme-scoped retrieval, action planning, dynamic knowledge structuring, and graph-augmented answering. Comprehensive experiments demonstrate RAS’s superior performance over existing approaches across diverse benchmarks. Looking ahead, RAS’s graph-based architecture enables seamless integration with external knowledge graphs and human-in-the-loop curation, making it particularly valuable for specialized domains requiring high accuracy and reliability. We discuss limitations of RAS and future work in Appendix A. Our code can be found at <https://anonymous.4open.science/r/RAS-Anonymous/>.

Impact Statements

The RAS framework has several potential positive impacts on both research and practical applications. First, by improving knowledge-intensive language generation through structured knowledge representation and iterative reasoning, RAS could enhance the reliability and factual accuracy of AI systems in critical domains like education, research assistance, and technical documentation. The theme-scoped retrieval mechanism also offers computational efficiency benefits, potentially reducing the energy consumption and computational resources required for large-scale knowledge retrieval tasks.

However, there are important considerations regarding potential risks and challenges. The framework’s effectiveness in knowledge structuring could potentially amplify existing biases in training data and knowledge sources. Additionally, while RAS improves retrieval efficiency, it may create barriers for low-resource languages or domains where structured knowledge bases and thematic classifications are limited. These concerns necessitate careful consideration of data sources and regular evaluation of the system’s impact across different user groups and application contexts.

References

- Anthropic. Introducing claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024. Accessed: 2025-01-06.
- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Es, S., James, J., Espinosa Anke, L., and Schockaert, S. RAGAs: Automated evaluation of retrieval augmented generation. In Aletras, N. and De Clercq, O. (eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150–158, St. Julians, Malta, March 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.eacl-demo.16/>.
- Fan, A., Jernite, Y., Perez, E., Grangier, D., Weston, J., and Auli, M. Eli5: Long form question answering. *arXiv preprint arXiv:1907.09190*, 2019.
- Gao, T., Yen, H., Yu, J., and Chen, D. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627*, 2023.
- Guan, J., Wu, W., Wen, Z., Xu, P., Wang, H., and Huang, M. Amor: A recipe for building adaptable modular knowledge agents through process feedback. *arXiv preprint arXiv:2402.01469*, 2024.
- Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.
- He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., and Hooi, B. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *arXiv preprint arXiv:2402.07630*, 2024.
- Ho, X., Nguyen, A.-K. D., Sugawara, S., and Aizawa, A. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.

- Hogan, A., Blomqvist, E., Cochez, M., d’Amato, C., Melo, G. D., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., et al. Knowledge graphs. *ACM Computing Surveys (Csur)*, 54(4):1–37, 2021.
- Izacard, G. and Grave, E. Leveraging passage retrieval with generative models for open domain question answering. In Merlo, P., Tiedemann, J., and Tsarfaty, R. (eds.), *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 874–880, Online, April 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.74. URL <https://aclanthology.org/2021.eacl-main.74>.
- Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., and Grave, E. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.
- Ji, Z., Yu, T., Xu, Y., Lee, N., Ishii, E., and Fung, P. Towards mitigating llm hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 1827–1843, 2023.
- Jiang, P., Cao, L., Xiao, C., Bhatia, P., Sun, J., and Han, J. Kg-fit: Knowledge graph fine-tuning upon open-world knowledge. In *The Thirty-Eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.
- Joshi, M., Choi, E., Weld, D. S., and Zettlemoyer, L. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Ju, M., Yu, W., Zhao, T., Zhang, C., and Ye, Y. Grape: Knowledge graph enhanced passage reader for open-domain question answering. *arXiv preprint arXiv:2210.02933*, 2022.
- Khattab, O., Santhanam, K., Li, X. L., Hall, D., Liang, P., Potts, C., and Zaharia, M. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp. *arXiv preprint arXiv:2212.14024*, 2023.
- Kim, D., Kang, D., Ryu, S., and Lee, G. G. Ontology-free general-domain knowledge graph-to-text generation dataset synthesis using large language model. *arXiv preprint arXiv:2409.07088*, 2024a.
- Kim, J., Nam, J., Mo, S., Park, J., Lee, S.-W., Seo, M., Ha, J.-W., and Shin, J. Sure: Summarizing retrievals using answer candidates for open-domain qa of llms. *arXiv preprint arXiv:2404.13081*, 2024b.
- Lehmann, J., Isele, R., Jakob, M., Jentzsch, A., Kontokostas, D., Mendes, P. N., Hellmann, S., Morsey, M., Van Kleef, P., Auer, S., et al. Dbpedia—a large-scale, multilingual knowledge base extracted from wikipedia. *Semantic web*, 6(2):167–195, 2015.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Lin, C.-Y. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.
- Ling, Z., Fang, Y., Li, X., Huang, Z., Lee, M., Memisevic, R., and Su, H. Deductive verification of chain-of-thought reasoning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Liu, X., Ji, K., Fu, Y., Tam, W. L., Du, Z., Yang, Z., and Tang, J. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021.
- Luo, H., Chuang, Y.-S., Gong, Y., Zhang, T., Kim, Y., Wu, X., Fox, D., Meng, H., and Glass, J. Sail: Search-augmented instruction learning. *arXiv preprint arXiv:2305.15225*, 2023.
- Lyu, Y., Li, Z., Niu, S., Xiong, F., Tang, B., Wang, W., Wu, H., Liu, H., Xu, T., and Chen, E. Crud-rag: A comprehensive chinese benchmark for retrieval-augmented generation of large language models, 2024a. URL <https://arxiv.org/abs/2401.17043>.
- Lyu, Y., Niu, Z., Xie, Z., Zhang, C., Xu, T., Wang, Y., and Chen, E. Retrieve-plan-generation: An iterative planning and answering framework for knowledge-intensive llm generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 4683–4702, 2024b.
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 2022.

- Maynez, J., Narayan, S., Bohnet, B., and McDonald, R. On faithfulness and factuality in abstractive summarization. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1906–1919, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.173. URL <https://aclanthology.org/2020.acl-main.173>.
- Min, S., Chen, D., Hajishirzi, H., and Zettlemoyer, L. A discrete hard em approach for weakly supervised question answering. *arXiv preprint arXiv:1909.04849*, 2019.
- Nussbaum, Z., Morris, J. X., Duderstadt, B., and Mulyar, A. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.
- Perozzi, B., Fatemi, B., Zelle, D., Tsitsulin, A., Kazemi, M., Al-Rfou, R., and Halcrow, J. Let your graph do the talking: Encoding structured data for llms. *arXiv preprint arXiv:2402.05862*, 2024.
- Pillutla, K., Swayamdipta, S., Zellers, R., Thickstun, J., Welleck, S., Choi, Y., and Harchaoui, Z. Mauve: Measuring the gap between neural text and human text using divergence frontiers. *Advances in Neural Information Processing Systems*, 34:4816–4828, 2021.
- Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., and Lewis, M. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- Reimers, N. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- Robertson, S., Zaragoza, H., et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., and Chen, W. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. *arXiv preprint arXiv:2305.15294*, 2023.
- Shi, Y., Huang, Z., Feng, S., Zhong, H., Wang, W., and Sun, Y. Masked label prediction: Unified message passing model for semi-supervised classification. *arXiv preprint arXiv:2009.03509*, 2020.
- Stelmakh, I., Luan, Y., Dhingra, B., and Chang, M.-W. Asqa: Factoid questions meet long-form answers. *arXiv preprint arXiv:2204.06092*, 2022.
- Sun, Y., Wang, S., Li, Y., Feng, S., Chen, X., Zhang, H., Tian, X., Zhu, D., Tian, H., and Wu, H. Ernie: Enhanced representation through knowledge integration. *arXiv preprint arXiv:1904.09223*, 2019.
- Wang, X., Wang, H., and Yang, D. Measure and improve robustness in nlp models: A survey. *arXiv preprint arXiv:2112.08313*, 2021.
- Xu, F., Shi, W., and Choi, E. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *arXiv:2210.03629*, 2023.
- Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. *arXiv preprint arXiv:2104.06378*, 2021.
- Yasunaga, M., Bosselut, A., Ren, H., Zhang, X., Manning, C. D., Liang, P. S., and Leskovec, J. Deep bidirectional language-knowledge graph pretraining. *Advances in Neural Information Processing Systems*, 35:37309–37323, 2022.
- Yu, W. Retrieval-augmented generation across heterogeneous knowledge. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Student Research Workshop*, pp. 52–58, 2022.
- Yu, W., Zhu, C., Li, Z., Hu, Z., Wang, Q., Ji, H., and Jiang, M. A survey of knowledge-enhanced text generation. *ACM Computing Survey (CSUR)*, 2022.
- Zhang, T., Luo, H., Chuang, Y.-S., Fang, W., Gaitskill, L., Hartvigsen, T., Wu, X., Fox, D., Meng, H., and Glass, J. Interpretable unified language checking. *arXiv preprint arXiv:2304.03728*, 2023a.
- Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., et al. Siren’s song in the ai ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023b.

Zhu, Y., Ren, C., Xie, S., Liu, S., Ji, H., Wang, Z., Sun, T.,
He, L., Li, Z., Zhu, X., and Pan, C. REALM: Rag-driven
enhancement of multimodal electronic health records
analysis via large language models. *arXiv:2402.07016*,
2024.

Contents of Appendix

A	Limitations & Future Work	14
B	HotpotQA-SUBQ Dataset	15
B.1	Supporting Document Filtering	15
B.2	Sub-query Generation	16
B.3	Sample Labeling	16
B.4	Dataset Statistics	18
B.5	Data Examples	18
C	Training Details	20
C.1	Classifier & Shifter Training for Theme Scoping	20
C.2	Text-to-Triples Model Training	21
C.3	Action Planner & Answerer Multi-task Training	23
D	Evaluation Datasets & Metrics	24
D.1	Test Datasets	24
D.2	Evaluation Metrics	25
E	Inference & Evaluation Details	26
E.1	Closed-Source Model Settings	26
E.2	Open-Source Model Settings	29
E.3	Instructions	29
F	Qualitative Examples	30
G	Hyperparameter Study	34
H	Notation Table	35

A. Limitations & Future Work

While RAS demonstrates strong performance, several key limitations warrant consideration for future improvements:

- **Knowledge Extraction Quality:** The effectiveness of both planning and answer generation depends on the quality of text-to-triple extraction. Despite using a Flan-T5-based model for triple extraction, implementing a more sophisticated approach – such as a dedicated Named Entity Recognition and Relation Extraction (NER-RE) pipeline – could potentially enhance RAS’s reasoning capabilities and knowledge representation.
- **Theme Scoping Limitations:** The current classifier was trained on DBPedia-298, an older dataset that may not adequately represent the thematic distribution of contemporary document collections. This temporal gap particularly affects performance when processing newer corpora, as evidenced by our experiments with Wikipedia 2020 dumps. The development of a huge, up-to-date multilabel document classification dataset would likely improve the effectiveness of theme scoping and, consequently, the overall efficiency gains it provides.
- **Graph Evolution Strategy:** The current approach to evolving knowledge graphs could be enhanced by incorporating more sophisticated graph pruning and merging strategies. Future work could explore dynamic graph summarization techniques to maintain the most relevant information while preventing excessive graph growth during iterations.
- **Cross-Domain Adaptability:** While RAS performs well on the evaluated tasks, its effectiveness across highly specialized domains or multilingual settings remains to be investigated. Future research could focus on developing domain-adaptation techniques and multilingual knowledge structuring approaches.
- **Interactive Refinement:** The current implementation lacks mechanisms for interactive refinement of the constructed knowledge graphs. Future versions could incorporate human feedback loops to improve the quality of knowledge structuring and reasoning paths.

These limitations suggest several promising directions for future research, including improved knowledge extraction techniques, more robust theme classification approaches, and enhanced graph evolution strategies. A particularly promising direction is the integration of external knowledge graphs into RAS’s iterative loop, as its graph-based architecture naturally accommodates the incorporation of established KGs like Wikidata or domain-specific knowledge bases. This could significantly enhance the system’s reasoning capabilities by combining dynamically constructed knowledge with curated structured information. We summarize the potential future directions of extension in Figure 5.

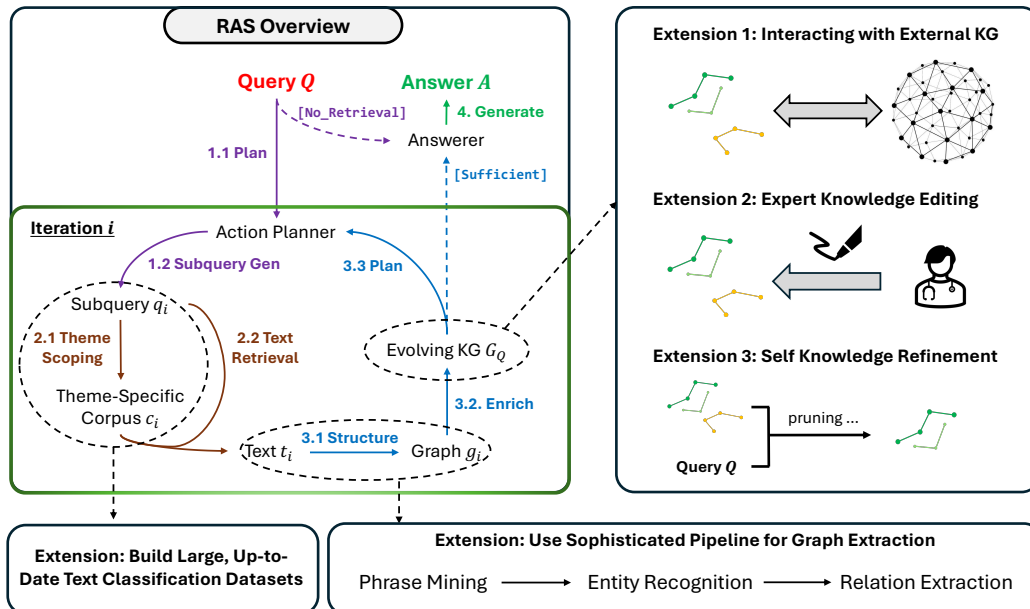


Figure 5. Potential Future Improvement of RAS.

B. HotpotQA-SUBQ Dataset

The source of our dataset is HotpotQA (Yang et al., 2018).² Our goal is to create a dataset that can let the model learn the capabilities of subquery generation and answering with intensive knowledge. We name the dataset as HotpotQA-SUBQ.

The construction of HotpotQA-SUBQ includes three steps: (1) filter out irrelevant/unnecessary “supporting docs” (§B.1), (2) generate one subquery for each supporting document (§B.2), and (3) sample labeling (§B.3).

B.1. Supporting Document Filtering

To identify and filter out irrelevant supporting documents from HotpotQA, we employ an instruction-based approach using Claude 3.5 Sonnet, as shown in Figure 6.

Identify which documents are HELPFUL to answer the question. Output only the document numbers separated by commas.

Examples:

Example 1 (Some documents are not helpful):

Question: What nationality was James Henry Miller's wife?

Supporting docs:

1. Margaret "Peggy" Seeger (born June 17, 1935) is an American folksinger. She is also well known in Britain, where she has lived for more than 30 years, and was married to the singer and songwriter Ewan MacColl until his death in 1989.

2. Seeger's father was Charles Seeger (1886–1979), an important folklorist and musicologist; her mother was Seeger's second wife, Ruth Porter Crawford.

3. James Henry Miller, better known by his stage name Ewan MacColl, was an English folk singer and songwriter.

Output: 1,3

Explanation: Only docs 1 and 3 are helpful - doc 1 shows Peggy Seeger (who is American) was married to Ewan MacColl, and doc 3 confirms Ewan MacColl is James Henry Miller. Doc 2 about Seeger's parents is not helpful.

Example 2 (All documents are helpful):

Question: The Oberoi family is part of a hotel company that has a head office in what city?

Supporting docs:

1. The Oberoi family is an Indian family that is famous for its involvement in hotels, namely through The Oberoi Group.

2. The Oberoi Group is a hotel company with its head office in Delhi.

Output: 1,2

Explanation: Both docs are helpful - doc 1 links the Oberoi family to The Oberoi Group, and doc 2 provides the head office location.

Question: [question]

Supporting docs:

[enumerated documents]

Output only the helpful document numbers separated by commas:

Figure 6. Prompt used for filtering supporting documents in HotpotQA. The prompt includes examples to demonstrate the difference between helpful and irrelevant documents. The input parts to the prompt are highlighted.

The filtering prompt is designed with clear examples that illustrate the criteria for document relevance. For each HotpotQA sample, we enumerate all supporting documents and use Claude to identify only those that contribute directly to answering the question. After filtering out irrelevant documents, we filter the question with no supporting documents. This filtering

²We use the version hotpot_train_v1.1 from <https://github.com/hotpotqa/hotpot>.

step reduces noise in the training data and helps focus the model on truly relevant information during sub-query generation.

B.2. Sub-query Generation

For sub-query generation, we use the template as follows to generate one subquery per (filtered) document:

Given this main question and a supporting document, generate a simple sub-query (a question) that will help retrieve information from the document to answer the main question.

Main Question: [main_question]

Current Document ([topic]):

[document_content]

[If previous queries exist:]

Previously generated sub-queries:

- [sub_query_1]

- [sub_query_2]

...

Write ONE clear and specific question that:

1. Can be answered using ONLY this document
2. Helps retrieve information needed for the main question
3. Is direct and focused on key information from this document

Write only the question, without any explanations or formatting.

Figure 7. Prompt used for subquery generation from HotpotQA. The input parts to the prompt are highlighted.

The prompt template enforces document-specificity, goal-orientation, and conciseness in sub-query generation. For iterative querying, we maintain a list of previously generated sub-queries to avoid redundancy and encourage progressive information gathering.

B.3. Sample Labeling

The sample labeling process transforms the filtered and subquery-augmented HotpotQA examples into training instances for both the Action Planner and Answerer components. We formalize this process in Algorithm B.3.

The algorithm takes as input the filtered HotpotQA dataset $\mathcal{D}$, base language model $\mathcal{M}$, and text-to-triple conversion model f_{t2t} . For each example, we first attempt direct answer generation using the base LLM (Line 3). If successful, we create [NO_RETRIEVAL] training instances for both the planner and answerer (Lines 4-6).

For examples requiring retrieval, we process each supporting document sequentially:

1. Convert document text to structured triples using f_{t2t} (Line 12)
2. For intermediate documents ($i < n$):
 - Construct input by concatenating previous subquery-graph pairs
 - Label with [SUBQ] and next subquery
3. For final document ($i = n$):

Algorithm 1 HotpotQA-SUBQ Sample Labeling

```

881 Require:  $\mathcal{D}$ : Filtered HotpotQA dataset
882 Require:  $\mathcal{M}$ : Base LLM (LLaMA-2-7B)
883 Require:  $f_{t2t}$ : Text-to-triple conversion model
884 Ensure:  $\mathcal{T}_{plan}$ : Training data for Action Planner
885 Ensure:  $\mathcal{T}_{ans}$ : Training data for Answerer
886 1: Initialize  $\mathcal{T}_{plan}, \mathcal{T}_{ans} \leftarrow \{\}, \{\}$ 
887 2: for all  $(Q, \{d_0, \dots, d_n\}, A) \in \mathcal{D}$  do
888 3:    $\hat{A} \leftarrow \mathcal{M}(Q)$   $\leftrightarrow$  Direct answer attempt
889 4:   if  $\hat{A} = A$  then
890 5:      $\mathcal{T}_{plan} \leftarrow \mathcal{T}_{plan} \cup \{(Q, [\text{NO\_RETRIEVAL}])\}$ 
891 6:      $\mathcal{T}_{ans} \leftarrow \mathcal{T}_{ans} \cup \{(Q, A)\}$ 
892 7:     continue
893 8:   end if
894 9:    $subq_0, \dots, subq_n \leftarrow \text{GenerateSubqueries}(Q, \{d_0, \dots, d_n\})$ 
895 10:   $G_0, \dots, G_n \leftarrow \emptyset$   $\leftrightarrow$  Initialize graph contexts
896 11:  for  $i \leftarrow 0$  to  $n$  do
897 12:     $g_i \leftarrow f_{t2t}(d_i)$   $\leftrightarrow$  Convert text to triples
898 13:    if  $i < n$  then
899 14:       $input \leftarrow \text{FormatInput}(\{(subq_j, g_j)\}_{j=0}^i, Q)$ 
900 15:       $\mathcal{T}_{plan} \leftarrow \mathcal{T}_{plan} \cup \{(input, [\text{SUBQ}] subq_{i+1})\}$ 
901 16:    else
902 17:       $input \leftarrow \text{FormatInput}(\{(subq_j, g_j)\}_{j=0}^n, Q)$ 
903 18:       $\mathcal{T}_{plan} \leftarrow \mathcal{T}_{plan} \cup \{(input, [\text{SUFFICIENT}])\}$ 
904 19:       $\mathcal{T}_{ans} \leftarrow \mathcal{T}_{ans} \cup \{(input, A)\}$ 
905 20:    end if
906 21:     $G_i \leftarrow G_{i-1} \cup g_i$   $\leftrightarrow$  Accumulate graph context
907 22:  end for
908 23: end for
909 24: return  $\mathcal{T}_{plan}, \mathcal{T}_{ans}$ 

```

- Include all accumulated context
- Label planner sample as [SUFFICIENT]
- Create answerer training instance with ground truth answer

The input formatting function (Lines 14 and 17) follows the template:

```

[SUBQ]  $q_0$ 
Retrieved Graph Information:  $g_0$ 
[SUBQ]  $q_1$ 
Retrieved Graph Information:  $g_1$ 
...
Question: Q

```

This process generates two datasets:

- $\mathcal{T}_{plan}$: Trains the Action Planner to determine retrieval needs and generate targeted subqueries
- $\mathcal{T}_{ans}$: Trains the Answerer to synthesize final responses from accumulated graph context

Table 3 illustrates the distribution of our generated datasets. The planner dataset demonstrates a balanced distribution across three label types: 35% [SUFFICIENT], 55% [SUBQ], and 10% [NO_RETRIEVAL]. The answerer dataset incorporates both no-retrieval and sufficient cases from multiple sources. The size of the answerer dataset exceeds the combined total of no-retrieval and sufficient cases from the planner dataset for two reasons: first, we incorporated additional data from ASQA and Arc-Easy datasets; second, we included no-retrieval cases that were initially filtered out in step 1 of our process. This comprehensive approach ensures a robust training set for the answerer component.

Our labeling approach ensures that models learn both the iterative nature of complex question answering and the importance of structured knowledge representation. The complementary training objectives help develop robust reasoning capabilities while maintaining retrievability of supporting evidence.

B.4. Dataset Statistics

		Planner Data	Answerer Data
# Queries		129,902	78,164 (w/ 3,897 ASQA & 2,147 Arc-Easy)
# Input Tokens	(Mean, Median, Max)	(338, 301, 1,910)	(475, 466, 2,214)
# Output Tokens	(Mean, Median, Max)	(13, 12, 62)	(13, 4, 2,332)
# Subqueries	(Min, Mean, Max)	(0, 0.8, 5)	(0, 1.2, 6)
# [SUFFICIENT]		45,722	–
# [SUBQ]		71,676	–
# [NO_RETRIEVAL]		12,504	–
# [SUBQ] Tokens	(Min, Mean, Max)	(6, 18.3, 62)	–
# Nodes	(Mean, Median, Max)	(11.6, 11, 56)	(11.7, 11, 84)
# Edges	(Mean, Median, Max)	(21.2, 20, 116)	(21.3, 20, 160)

Table 3. Statistics of our constructed HotpotQA-SUBQ dataset.

B.5. Data Examples

"input": "
 You are a planner to determine if the question can be answered with current information and output the appropriate label as well as the subquery if needed.
 Output [NO_RETRIEVAL] if the question can be directly answered with the question itself without any retrieval.
 Output [SUBQ] with an subquery for retrieval if still needs a subquery.
 Output [SUFFICIENT] if the question can be answered with the provided information.

Question: Given a chat history separated by new lines, generates an informative, knowledgeable and engaging response. ## Input:\n\nI love pizza. While it's basically just cheese and bread you can top a pizza with vegetables, meat etc. You can even make it without cheese!\nPizza is the greatest food ever! I like the New York style.\nI do too. I like that the crust is only thick and crisp at the edge, but soft and thin in the middle so its toppings can be folded in half.\nAbsolutely! I am not that big of a fan of Chicago deep dish though.",
"label": "[NO_RETRIEVAL]"

Figure 8. Training Data Example (Action Planner – [NO_RETRIEVAL])

"input": "
 You are a planner to determine if the question can be answered with current information and output the appropriate label as well as the subquery if needed.
 Output [NO_RETRIEVAL] if the question can be directly answered with the question itself without any retrieval.
 Output [SUBQ] with an subquery for retrieval if still needs a subquery.
 Output [SUFFICIENT] if the question can be answered with the provided information.

[SUBQ] Where is The Pick Motor Company Limited located?
Retrieved Graph Information: [(S> Pick Motor Company Limited| P> Alias| O> New Pick Motor Company)', '(S> Pick Motor Company Limited| P> Location| O> Stamford, Lincolnshire)', '(S> Pick Motor Company Limited| P> Operational period| O> 1899-1925)', '(S> Pick Motor Company Limited| P> Industry| O> Motor vehicle manufacturing)']

Question: The Pick Motor Company Limited is located in a town on which river ?",
"label": "[SUBQ] Which river is Stamford located on?"

Figure 9. Training Data Example (Action Planner – [SUBQ])

"input": "
 You are a planner to determine if the question can be answered with current information and output the appropriate label as well as the subquery if needed.
 Output [NO_RETRIEVAL] if the question can be directly answered with the question itself without any retrieval.
 Output [SUBQ] with an subquery for retrieval if still needs a subquery.
 Output [SUFFICIENT] if the question can be answered with the provided information.

[SUBQ] What gaming control board in Ohio is Martin R. Hoke a member of?
Retrieved Graph Information: ['(S> Martin R. Hoke| P> Former position| O> Member of the United States House of Representatives)', '(S> Martin R. Hoke| P> Birth date| O> May 18, 1952)', '(S> Martin R. Hoke| P> Occupation| O> Politician)', '(S> Martin R. Hoke| P> State| O> Ohio)', '(S> Martin R. Hoke| P> Nationality| O> American)', '(S> Martin R. Hoke| P> Party| O> Republican)', '(S> Martin R. Hoke| P> Member of| O> Ohio Casino Control Commission)', '(S> Martin R. Hoke| P> Born| O> 1952)']

[SUBQ] What gaming control board provides oversight of Ohio's casinos?
Retrieved Graph Information: ['(S> Ohio Casino Control Commission| P> Function| O> Provides oversight of the state's casinos)', '(S> Ohio Casino Control Commission| P> Location| O> Ohio)', '(S> Ohio Casino Control Commission| P> Type| O> Gaming control board)', '(S> Ohio Casino Control Commission| P> Abbreviation| O> OCCC)']

Question: Martin R. Hoke, is an American Republican politician, member of which gaming control board in Ohio that provides oversight of the state's casinos?
 ",

"label": "[SUFFICIENT]"

Figure 10. Training Data Example (Action Planner – [SUFFICIENT])

"input": "
 You are a answerer given a question and retrieved graph information.
 Each [SUBQ] is a subquery we generated through reasoning for the question. The retrieved graph information follows each [SUBQ] is relevant graph information we retrieved to answer the subquery.
 [NO_RETRIEVAL] means the question can be answered with the question itself without any retrieval.
 The main question starts with \"Question: \". Please answer the question, with subqueries and retrieved graph information if they are helpful.

[NO_RETRIEVAL]
Question: Which person won the Nobel Prize in Literature in 1961, Ivo Andri or Nicholas Pileggi?",

"label": "Ivo Andri"

Figure 11. Training Data Example (Answerer – [NO_RETRIEVAL])

"input": "
 You are a answerer given a question and retrieved graph information.
 Each [SUBQ] is a subquery we generated through reasoning for the question. The retrieved graph information follows each [SUBQ] is relevant graph information we retrieved to answer the subquery.
 [NO_RETRIEVAL] means the question can be answered with the question itself without any retrieval.
 The main question starts with \"Question: \". Please answer the question, with subqueries and retrieved graph information if they are helpful.

[SUBQ] What type of athlete is Darold Williamson?
Retrieved Graph Information: ['(S> Darold Williamson| P> Nationality| O> American)', '(S> Darold Williamson| P> Birth date| O> February 19, 1983)', '(S> Darold Williamson| P> Occupation| O> Track athlete)']

[SUBQ] What specific skills are included in track and field events?
Retrieved Graph Information: ['(S> Athletics| P> Includes| O> Road running)', '(S> Track and field| P> Includes| O> Throwing)', '(S> Track and field| P> Includes| O> Jumping)', '(S> Athletics| P> Includes| O> Cross country running)', '(S> Track and field| P> Categorised under| O> Athletics)', '(S> Track and field| P> Includes| O> Running)', '(S> Track and field| P> Venue| O> Stadium with an oval running track enclosing a grass field)', '(S> Athletics| P> Includes| O> Track and field)', '(S> Athletics| P> Includes| O> Race walking)', '(S> Track and field| P> Based on skills of| O> Running)', '(S> Track and field| P> Based on skills of| O> Jumping)', '(S> Track and field| P> Based on skills of| O> Throwing)']

Question: Darold Williamson is an athlete in what running and jumping sport?",

"label": "Track and field"

Figure 12. Training Data Example (Answerer – [SUFFICIENT])

C. Training Details

C.1. Classifier & Shifter Training for Theme Scoping

Classifier Training. For theme classification, we trained a multilabel document classifier on the DBPedia-298 dataset (Lehmann et al., 2015) using a two-stage architecture. The first stage uses the nomic-embed-text-v1 encoder model (Nussbaum et al., 2024) to generate document embeddings, followed by a classification head consisting of a two-layer neural network. The training size and the validation size are 240,932 and 36,008, respectively.

The classifier consists of:

- An encoder based on nomic-embed-text-v1 for generating document embeddings
- A classification head with:
 - Input layer: embedding dimension $\rightarrow$ 512 units
 - ReLU activation
 - Dropout ($p=0.1$)
 - Output layer: 512 $\rightarrow$ number of labels (298)
 - Sigmoid activation for multilabel outputs

We jointly trained both the encoder and classification head with different learning rates:

- **Classification head:** Trained with a higher learning rate ($2e-4$) to allow faster adaptation to the classification task
- **Text Encoder:** Fine-tuned with a lower learning rate ($2e-5$) to preserve pretrained knowledge while making subtle adjustments for the domain

This dual learning rate strategy allows the model to benefit from the pretrained knowledge in the encoder while optimizing the classification head for the specific multilabel task. The lower learning rate for the encoder helps prevent catastrophic forgetting of the pretrained representations.

The model was evaluated using micro-averaged metrics on the validation set during training, with monitoring of F1 score, precision, and recall, as shown in Figure 14. We select the best classifier model based on the best validation F1 score achieved during training.

Distribution Shifter Training. The theme distribution shifter maps query theme distributions to expected document theme distributions. Training involves two phases:

Phase 1: PPD Labeling Using our constructed HotpotQA-SUBQ dataset (in Appendix B) with 204,638 query-doc pairs (where we set training-validation ratio as 9:1):

- **Input Generation:** Process all subqueries and supporting documents through the trained theme classifier to obtain their Posterior Probability Distributions (PPDs)
- **Pairing:** Each subquery is paired with its corresponding supporting document, creating query-document PPD pairs
- **Data Structure:** The labeled dataset consists of:
 - Input: Query PPDs (298-dimensional probability vectors)
 - Target: Corresponding document PPDs (298-dimensional probability vectors)

Phase 2: Shifter Training Train a neural network to map query PPDs to document PPDs:

- **Architecture:** A three-layer MLP with:
 - Input layer: 298 $\rightarrow$ 512 units
 - Hidden layer: 512 $\rightarrow$ 256 units

- Output layer: $256 \rightarrow 298$ units
- ReLU activations and dropout ($p=0.2$) between layers
- Softmax activation for final output
- **Training Objectives:** Multiple distribution alignment metrics:
 - KL divergence loss (primary objective)
 - Jensen-Shannon divergence monitoring
 - Wasserstein distance monitoring
 - L1 and L2 distance monitoring
- **Training Configuration:**
 - Optimizer: Adam with learning rate $1e-3$
 - Batch size: 32
 - Early stopping based on validation loss
 - Numerical stability ensured through epsilon addition ($1e-15$)

Performance is tracked using validation KL divergence, Jensen-Shannon divergence, and Wasserstein distance to ensure proper distribution alignment, as shown in Figure 15.

C.2. Text-to-Triples Model Training

For our text-to-triple conversion model, we fine-tuned a Flan-T5-Large model (Chung et al., 2024) to transform raw text passages into structured knowledge triples. The model processes input text sequences up to 512 tokens in length and generates structured triples in a standardized format “(S > subject| P> predicate| O> object)”.

Our training dataset WikiOFGraph (Kim et al., 2024a), curated by ChatGPT, comprises 5,851,776 text-triple pairs, with an additional 5,000 samples reserved for validation. Each training instance consists of a natural language text passage paired with its corresponding set of comma-separated triples. For example:

Text:
 "William Gerald Standridge (November 27, 1953 – April 12, 2014) was an American stock car racing driver. He was a competitor in the NASCAR Winston Cup Series and Busch Series."

Triples:
 (S> William gerald standridge| P> Nationality| O> American),
 (S> William gerald standridge| P> Occupation| O> Stock car racing driver),
 (S> William gerald standridge| P> Competitor| O> Busch series),
 (S> William gerald standridge| P> Competitor| O> Nascar winston cup series),
 (S> William gerald standridge| P> Birth date| O> November 27, 1953),
 (S> William gerald standridge| P> Death date| O> April 12, 2014)

Figure 13. Example of WikiOFGraph data

We implemented the training using the AdamW optimizer with a learning rate of $2e-5$, incorporating linear warmup and decay schedules. The training process ran for 500,000 steps with a batch size of 32 per GPU and gradient accumulation over 4 steps. To optimize training efficiency and memory usage, we employed mixed-precision training using bfloat16 and applied weight decay at 0.01. The maximum source and target sequence lengths were set to 512 and 256 tokens respectively.

For evaluation, we primarily relied on ROUGE-1, ROUGE-2, and ROUGE-L scores to assess the quality of triple generation. We supplemented these metrics with custom triple matching accuracy measures that consider subject matching, predicate normalization, and object entity alignment. Validation metrics were computed at 5,000-step intervals throughout training.

The training process followed a curriculum learning approach, beginning with shorter, simpler text passages and progressively introducing longer and more complex examples. We implemented early stopping based on validation ROUGE-L scores and maintained model checkpoints at points of peak validation performance. To prevent gradient instability, we applied gradient clipping with a threshold of 1.0. The training curves for various metrics are shown in Figure 16, demonstrating steady improvement in the model’s ability to extract structured knowledge from text. The training of this model was conducted on eight NVIDIA RTX 6000 with 48GB memory.

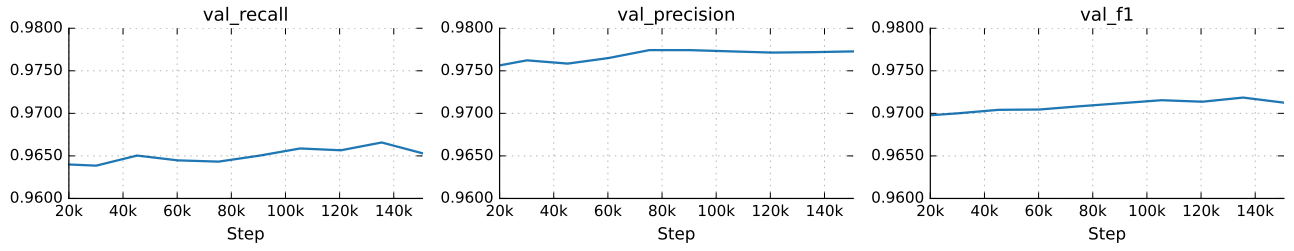


Figure 14. **Multilabel Document Classifier Training.** It shows the training metrics for the multilabel document classifier, including validation precision, recall, and F1 score.

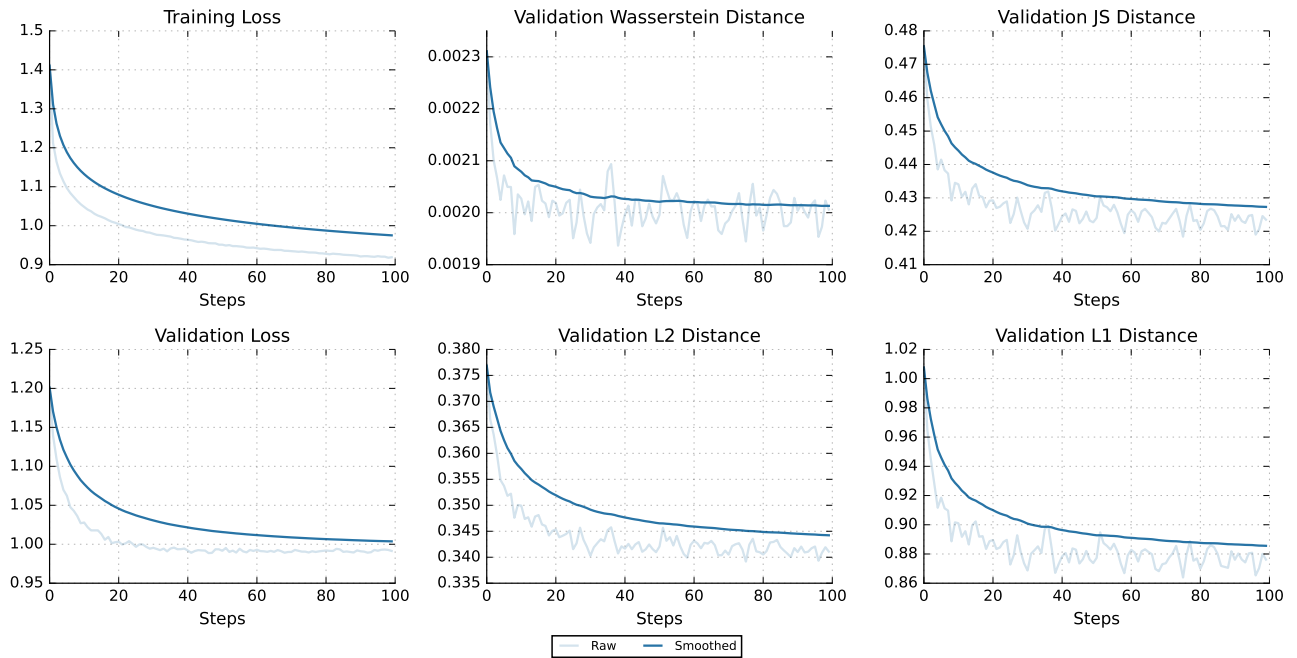


Figure 15. **Distribution Shifter Training.** It displays various distance metrics used to evaluate the quality of distribution alignment.

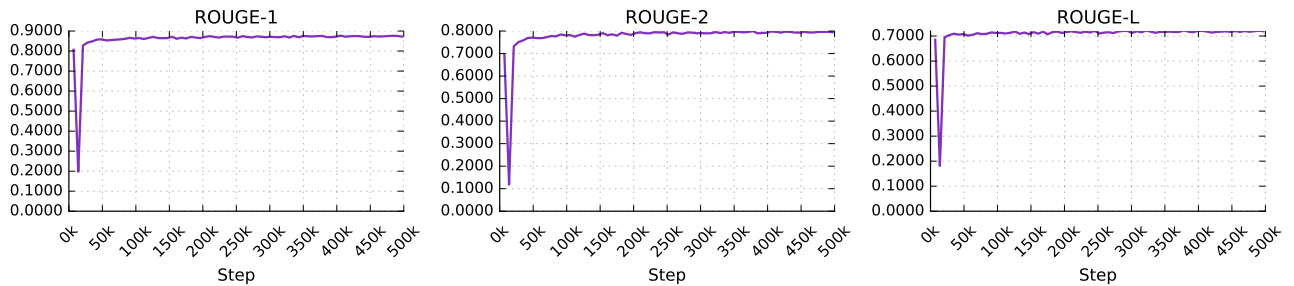


Figure 16. **Text-to-Triple Model Training.** It shows the validation rouge scores to evaluate the extracted triple list.

C.3. Action Planner & Answerer Multi-task Training

We employ GraphLLM (Perozzi et al., 2024; He et al., 2024) as our foundational model architecture and utilize our constructed HotpotQA-SUBQ dataset to train a unified model capable of performing both graph-conditioned action planning and answer generation through multitask learning.

Since the Action Planner and Answerer share an identical architectural foundation, as illustrated in Figure 1, we differentiate their functionality through specialized instruction sets. For the Action Planner $\mathcal{M}_{Plan}$, we employ the following instruction template:

You are a planner to determine if the question can be answered with current information and output the appropriate label as well as the subquery if needed.
 Output [NO RETRIEVAL] if the question can be directly answered with the question itself without any retrieval.
 Output [SUBQ] with an subquery for retrieval if still needs a subquery.
 Output [SUFFICIENT] if the question can be answered with the provided information.

Figure 17. Instruction $INST_{plan}$ for Action Planner $\mathcal{M}_{Plan}$.

For the Answerer $\mathcal{M}_{Ans}$, we utilize this distinct instruction set:

You are an answerer given a question and retrieved graph information.
 Each [SUBQ] is a subquery we generated through reasoning for the question. The retrieved graph information follows each [SUBQ] is relevant graph information we retrieved to answer the subquery.
 [NO RETRIEVAL] means the question can be answered with the question itself without any retrieval.
 The main question starts with "Question: ". Please answer the question, with subqueries and retrieved graph information if they are helpful.

Figure 18. Instruction $INST_{ans}$ for Answer $\mathcal{M}_{Ans}$.

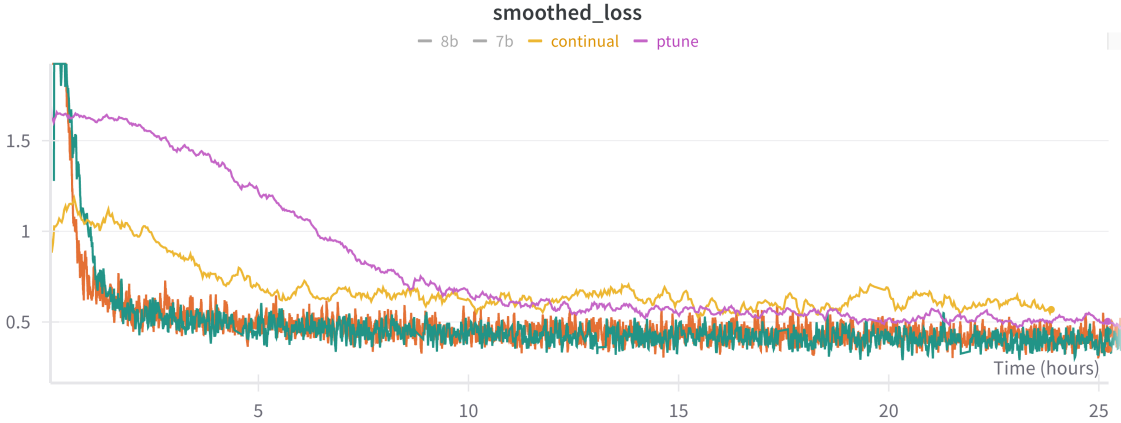


Figure 19. Training loss comparison over one epoch. The plot compares RAS-7B (green) and RAS-8B (orange) training trajectories. Two additional RAS-7B variants are shown: “continual”: a continual learning approach where Answerer training precedes Action Planner training, and “ptune”: a parameter-efficient variant that only tunes graph tokens without LoRA. The lower and more stable loss curves of the standard RAS variants demonstrate the effectiveness of joint training with LoRA. We use the last 100 steps for loss smoothing.

For graph encoding, we implement Graph Transformer (Shi et al., 2020), selected for its robust capability in handling non-fully-connected graphs—a common characteristic of text-extracted triples. Our base language models comprise

LLaMA-2-7B and LLaMA-3-8B, chosen both to maintain consistency with previous research (Asai et al., 2023; Lyu et al., 2024b) and to investigate our framework’s performance scaling across different model capacities.

Our implementation of GraphLLM differs from G-Retriever (He et al., 2024) primarily due to the distinct nature of our graph structures. While G-Retriever operates on single interconnected graphs, our framework processes multiple potentially disconnected subgraphs, each corresponding to different subqueries. To address this architectural difference, we adopt a sequential encoding strategy: rather than encoding the entire graph at once, we process each subgraph individually using Graph Transformer, followed by mean pooling across all subgraph embeddings to produce the final encoded representation.

The training process utilizes 4 NVIDIA RTX 6000 Ada Generation GPUs, each with 48GB memory. We train all models for 2 epochs using a batch size of 2 and gradient accumulation steps of 2, implementing a peak learning rate of $1e-5$ with a 0.15 warmup ratio and 0.01 decay rate. The maximum sequence lengths are set to 300 tokens for generation and 2,500 tokens for input, with training conducted in BFloat16 precision.

D. Evaluation Datasets & Metrics

D.1. Test Datasets

We evaluate RAS on diverse benchmark datasets spanning short-form QA, closed-set tasks, and long-form generation in the zero-shot setting, aligning with (Asai et al., 2023; Lyu et al., 2024b). Below we describe each dataset:

Short-form Generation Datasets:

- **TriviaQA-unfiltered (TQA)** (Joshi et al., 2017): A large-scale QA dataset containing 11,313 question-answer pairs in our test set. The questions are sourced from trivia enthusiasts and cover diverse topics.
- **2WikiMultiHopQA (2WQA)** (Ho et al., 2020): A multi-hop question answering dataset (with 12,576 samples in test set) that requires models to combine information from multiple Wikipedia articles to answer questions.
- **PopQA** (Mallen et al., 2022): A dataset focusing on questions about long-tail entities, containing 1,399 queries where the monthly Wikipedia page views are less than 100. These questions test models’ ability to handle queries about less common entities.

Closed-set Tasks:

- **PubHealth** (Pub) (Zhang et al., 2023a): A dataset for verifying health-related claims. Models must classify statements as “true” or “false” based on scientific evidence. The dataset contains 987 test samples.
- **ARC-Challenge** (ARC) (Clark et al., 2018): A multiple-choice science question dataset designed to be challenging for models, requiring multi-hop reasoning and background knowledge. The dataset contains 1,172 test samples.

Long-form Generation Datasets:

- **ASQA** (Stelmakh et al., 2022): A long-form question answering dataset that requires models to generate comprehensive answers with proper citation of sources. Models must balance information completeness with factual accuracy. The dataset contains 948 test samples.
- **ELI5** (Fan et al., 2019): A dataset derived from the “Explain Like I’m Five” subreddit, containing questions seeking straightforward explanations of complex topics. The dataset contains 1,000 test samples.

D.2. Evaluation Metrics

We employ different evaluation metrics appropriate for each task category:

Short-form Generation:

- For PopQA and TriviaQA, we use the "golden match" metric (Asai et al., 2023; Min et al., 2019; Guu et al., 2020)³, where a prediction p is considered correct if it contains any normalized version of the ground truth answers $G = \{g_1, \dots, g_n\}$:

$$\text{match}(p, G) = \begin{cases} 1 & \text{if } \exists g \in G : \text{norm}(g) \subseteq \text{norm}(p) \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

where $\text{norm}(\cdot)$ normalizes text by lowercasing, removing articles and punctuation.

- For 2WikiMultiHopQA, we follow RPG (Lyu et al., 2024b)⁴ to use token-level F1 score between prediction p and ground truth g :

$$\text{F1}(p, g) = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad (13)$$

where precision and recall are computed over normalized token overlap:

$$\text{precision} = \frac{|\text{tokens}(p) \cap \text{tokens}(g)|}{|\text{tokens}(p)|} \quad (14)$$

$$\text{recall} = \frac{|\text{tokens}(p) \cap \text{tokens}(g)|}{|\text{tokens}(g)|} \quad (15)$$

Closed-set Tasks: For both PubHealth and ARC-Challenge, we use accuracy (Asai et al., 2023), computed as:

$$\text{accuracy} = \frac{|\{i : \text{norm}(p_i) = \text{norm}(g_i)\}|}{N} \times 100 \quad (16)$$

where N is the total number of examples.

Long-form Generation: For ASQA and ELI5, we use multiple metrics⁵:

- ROUGE-L score to measure the longest common subsequence between prediction and reference (Lin, 2004)
- MAUVE score (Pillutla et al., 2021) to test the generation fluency by comparing the distribution of generated text against human references

All scores are reported as percentages, multiplied by 100.

³Implementation aligns with https://github.com/AkariAsai/self-rag/blob/main/retrieval_lm/metrics.py

⁴Implementation aligns with <https://github.com/haruhi-sudo/RPG/blob/main/retriever/src/evaluation.py>

⁵Implementation aligns with <https://github.com/princeton-nlp/ALCE/blob/main/eval.py>

E. Inference & Evaluation Details

We evaluated RAS using both closed-source and open-source language models. For closed-source evaluation, we used Claude-3.5-Sonnet (Sonnet-3.5) as our base model. For open-source evaluation, we tested with both LLaMA-2-7B and LLaMA-3-8B, as shown in the performance table (Table 1).

For the open-source models, we first fine-tuned the GraphLLM architecture on our HotpotQA-SUBQ dataset (see Section C.3). We then conducted zero-shot knowledge-intensive generation tests. With Claude-3.5-Sonnet, we used few-shot prompting for both action planning and answer generation phases. Below, we detail our approach for each model type.

E.1. Closed-Source Model Settings

For Text-to-Triple Conversion (Figure 20), we instruct the model to extract structured knowledge triples following specific formatting rules. The prompt precisely defines the triple format as (*S* > *subject* | *P* > *predicate* | *O* > *object*) and provides comprehensive guidelines to ensure consistent knowledge representation. Key rules include extracting maximal meaningful relationships, maintaining original entity casing, avoiding pronoun usage, and ensuring clear predicate specification. The example demonstrates the conversion of a biographical text into structured triples, showing how complex information about an individual’s life, career, and temporal details can be systematically decomposed into atomic facts. This standardized format enables reliable knowledge accumulation and reasoning in subsequent stages of the RAS framework.

Extract relationship triples from the given text.
Each triple should have exactly one subject (S>), one predicate (P>), and one object (O>).

Rules:

1. Extract as many meaningful triples as possible
2. Each triple must be in format: (S> subject| P> predicate| O> object)
3. Multiple triples should be separated by commas
4. Avoid using pronouns (it/he/she) - always use the actual names
5. Keep all entities in their original case (uppercase/lowercase)
6. Make predicates clear and specific
7. When input is only an entity, output (S> ENTITY| P> is| O> ENTITY) where ENTITY is the entity in the input.
8. [IMPORTANT] Do not include any other text in the output, only the triples or the entity (for the Rule 7 case).

Example Input:
"William Gerald Standridge (November 27, 1953 – April 12, 2014) was an American stock car racing driver. He was a competitor in the NASCAR Winston Cup Series and Busch Series."

Example Output:
(S> William gerald standridge| P> Nationality| O> American),
(S> William gerald standridge| P> Occupation| O> Stock car racing driver),
(S> William gerald standridge| P> Competitor| O> Busch series),
(S> William gerald standridge| P> Competitor| O> Nascar winston cup series),
(S> William gerald standridge| P> Birth date| O> November 27, 1953),
(S> William gerald standridge| P> Death date| O> April 12, 2014)

Input Text:
{text}

Figure 20. Few-shot prompt for text-to-triples transformation with closed-source LLM.

For Action Planning (Figure 21), we design a comprehensive prompt that guides the model in making strategic decisions about information retrieval needs. The prompt instructs the model to output one of three labels based on careful analysis of the current knowledge state: (1) [NO.RETRIEVAL] when the question can be answered directly, either due to the model’s inherent knowledge or the question’s nature, (2) [SUBQ] accompanied by a focused subquery when additional information is needed, or (3) [SUFFICIENT] when the accumulated knowledge is adequate to answer the question. The prompt includes diverse examples demonstrating different scenarios:

- (1) Generating follow-up queries based on partial information
- (2) Creating new queries when relevant information is missing
- (3) Recognizing when accumulated information is sufficient
- (4) Identifying questions that don’t require external knowledge
- (5) Handling common knowledge questions

A key feature of the prompt is its emphasis on query efficiency - it explicitly prohibits generating redundant subqueries that might retrieve already-known information. This design helps maintain the system’s efficiency while ensuring comprehensive knowledge gathering.

You are a planner to determine if the question can be answered with current information (Subquery [PREV_SUBQ] and retrieved graph information [PREV_GRAPH_INFO]) and output the appropriate label as well as the subquery if needed.
Output [NO_RETRIEVAL] if the question can be directly answered with the question itself without any retrieval. You are expected to output [NO_RETRIEVAL] either if you believe an LLM is knowledgeable enough to answer the question, or if you believe the question type is not suitable for retrieval.
Output [SUBQ] with an subquery for retrieval if still needs a subquery. Do not make an similar subquery that has been made before ([PREV_SUBQ]), as it is very likely to retrieve the same information.
Output [SUFFICIENT] if the question can be answered with the provided information.
The main question starts with "Question: ".

Examples:

Example 1 (Use the information in [PREV_GRAPH_INFO] to further generate a new subquery):
Input:
[PREV_SUBQ] Where is The Pick Motor Company Limited located?
[PREV_GRAPH_INFO] ['(S> Pick Motor Company Limited| P> Alias| O> New Pick Motor Company)', '(S> Pick Motor Company Limited| P> Location| O> Stamford, Lincolnshire)', '(S> Pick Motor Company Limited| P> Operational period| O> 1899-1925)', '(S> Pick Motor Company Limited| P> Industry| O> Motor vehicle manufacturing)']
Question: The Pick Motor Company Limited is located in a town on which river ?,
Output:
[SUBQ] Which river is Stamford located on?

Example 2 (No relevant information found in [PREV_GRAPH_INFO]):
Input:
[PREV_SUBQ] What medals did Michael Johnson win in the 1996 Olympics?
[PREV_GRAPH_INFO] ['(S> Michael Johnson| P> Nationality| O> American)', '(S> Michael Johnson| P> Birth date| O> September 13, 1967)', '(S> Michael Johnson| P> Sport| O> Track and field)', '(S> Michael Johnson| P> Team| O> United States Olympic team)']
Question: What was Michael Johnson's winning time in the 400m at the 1996 Olympics?
Output:
[SUBQ] What records or times did Michael Johnson set in the 400m at the 1996 Olympic Games?

Example 3 (The current information is sufficient to answer the question):
Input:
[PREV_SUBQ] What gaming control board in Ohio is Martin R. Hoke a member of?
[PREV_GRAPH_INFO] ['(S> Martin R. Hoke| P> Former position| O> Member of the United States House of Representatives)', '(S> Martin R. Hoke| P> Birth date| O> May 18, 1952)', '(S> Martin R. Hoke| P> Occupation| O> Politician)', '(S> Martin R. Hoke| P> State| O> Ohio)', '(S> Martin R. Hoke| P> Nationality| O> American)', '(S> Martin R. Hoke| P> Party| O> Republican)', '(S> Martin R. Hoke| P> Member of| O> Ohio Casino Control Commission)', '(S> Martin R. Hoke| P> Born| O> 1952)']
[PREV_SUBQ] What gaming control board provides oversight of Ohio's casinos?
[PREV_GRAPH_INFO] ['(S> Ohio Casino Control Commission| P> Function| O> Provides oversight of the state's casinos)', '(S> Ohio Casino Control Commission| P> Location| O> Ohio)', '(S> Ohio Casino Control Commission| P> Type| O> Gaming control board)', '(S> Ohio Casino Control Commission| P> Abbreviation| O> OCCC)']
Question: Martin R. Hoke, is an American Republican politician, member of which gaming control board in Ohio that provides oversight of the state's casinos?
Output:
[SUFFICIENT]

Example 4 (The question is not suitable for retrieval):
Input:
Given a chat history separated by new lines, generates an informative, knowledgeable and engaging response.
##Input:
I love pizza. While it's basically just cheese and bread you can top a pizza with vegetables, meat etc. You can even make it without cheese!
Pizza is the greatest food ever! I like the New York style.
I do too. I like that the crust is only thick and crisp at the edge, but soft and thin in the middle so its toppings can be folded in half.
Absolutely! I am not that big of a fan of Chicago deep dish though
Output:
[NO_RETRIEVAL]

Example 5 (You are knowledgeable enough to answer the question):
Input:
What is the capital of the United States?
Output:
[NO_RETRIEVAL]

[VERY IMPORTANT] Please only either output (1) [NO_RETRIEVAL] or (2) [SUBQ] with an concrete subquery for retrieval, or (3) [SUFFICIENT] if the question can be answered with the provided information.
[VERY IMPORTANT] Do not output any other text. DO NOT make an identical subquery [SUBQ] that has been made before ([PREV_SUBQ])!

Now, your turn:
Input:
{planner_input}
Output:

Figure 21. Few-shot prompt for action planning with closed-source LLM.

For Answer Generation (Figure 22), we design a prompt that focuses on synthesizing precise answers from structured knowledge graphs. The prompt emphasizes selective use of retrieved information - instructing the model to utilize subqueries and graph information only when relevant to the question at hand. A distinctive feature of this prompt is its requirement for definitive answers even under uncertainty or incomplete information, ensuring the model always provides a response.

The prompt includes carefully selected examples demonstrating two key scenarios: direct fact retrieval (Nobel Prize winner) and complex reasoning across multiple knowledge pieces (athlete's sport classification). These examples illustrate how to effectively combine information from multiple subqueries and their associated graph information to construct accurate answers. The prompt strictly enforces concise answer generation, requiring only the essential information without additional explanation or commentary.

You are a answerer given a question and retrieved graph information.
Each [SUBQ] is a subquery we generated through reasoning for the question. The retrieved graph information follows each [SUBQ] is relevant graph information we retrieved to answer the subquery.
The main question starts with "Question: ". Please answer the question, with subqueries and retrieved graph information if they are helpful (do not use them if they are not helpful).
You must answer the question, even if there's no enough information to answer the question, or you are not sure about the answer.

Examples:

Example 1:

Input:

Question: Which person won the Nobel Prize in Literature in 1961, Ivo Andri or Nicholas Pileggi?",

Output:

Ivo Andri

Example 2:

[SUBQ] What type of athlete is Darold Williamson?

Retrieved Graph Information: ['(S> Darold Williamson| P> Nationality| O> American)', '(S> Darold Williamson| P> Birth date| O> February 19, 1983)', '(S> Darold Williamson| P> Occupation| O> Track athlete)']

[SUBQ] What specific skills are included in track and field events?

Retrieved Graph Information: ['(S> Athletics| P> Includes| O> Road running)', '(S> Track and field| P> Includes| O> Throwing)', '(S> Track and field| P> Includes| O> Jumping)', '(S> Athletics| P> Includes| O> Cross country running)', '(S> Track and field| P> Categorised under| O> Athletics)', '(S> Track and field| P> Includes| O> Running)', '(S> Track and field| P> Venue| O> Stadium with an oval running track enclosing a grass field)', '(S> Athletics| P> Includes| O> Track and field)', '(S> Athletics| P> Includes| O> Race walking)', '(S> Track and field| P> Based on skills of| O> Running)', '(S> Track and field| P> Based on skills of| O> Jumping)', '(S> Track and field| P> Based on skills of| O> Throwing)']

Question: Darold Williamson is an athlete in what running and jumping sport?",

Output:

Track and field

[VERY IMPORTANT] Please only output the answer to the question.

[VERY IMPORTANT] Do not output any other text.

Now, your turn:

Input:

{answerer_input}

Output:

Figure 22. Few-shot prompt for answer generation with closed-source LLM.

In addition, for ASQA and ELI5 with closed-source models (RAS_{Sonnet-3.5} and all the other baselines (e.g., SuRe) using Sonnet-3.5), we conduct few-shot inference with two in-context learning demonstrations, aligning with previous study's (Gao et al., 2023) implementation.⁶⁷

⁶<https://github.com/princeton-nlp/ALCE/blob/main/run.py>

⁷Exemplars can be found at <https://github.com/princeton-nlp/ALCE/tree/main/prompts>

E.2. Open-Source Model Settings

For inference with open-source model, we use our GraphLLM trained by Hotpot-SUBQ (see Appendix C.3), we use 8 NVIDIA RTX 6000 Ada Generation with 48GB memory and CUDA version 12.4. We use Python 3.10, PyTorch 2.1.2, and torch-geometric 2.6.1 throughout all experiments.

For more efficient text retrieval, we split both theme index and dense index of the corpus into eight splits, and load them on eight GPUs with faiss-gpu 1.7.2.⁸

We set maximum new tokens as **100 for PopQA, TriviaQA, and 2WikiMultihopQA**, as **50 for PubHealth and ARC-Challenge**, and as **300 for ASQA and ELI5**, aligning with previous study (Asai et al., 2023; Lyu et al., 2024b). All the generation are configured with **batch size of 1**.

As we always set the main query as the first subquery, we pre-extract the triples from the context at the first iteration using our Flan-T5-based text-to-triple model for batch triple extraction. Notably, when we experimented with using Claude 3.5 Sonnet for triple extraction using the prompt from Figure 20, we observed an additional 3% average performance gain. This suggests room for improvement in our triple extraction approach - one potential direction would be training a larger model like LLaMA-2-7B on the WikiOFGraph dataset to achieve higher-quality triple extraction.

We use exact same instructions for action planner and answerer as we used in training phase, as shown in Figures 17 and 18.

E.3. Instructions

We use the following task-specific instructions for zero-shot inference:

Dataset	Instruction
ARC-C (baseline)	Given four answer candidates, A, B, C and D, choose the best answer choice. Please answer with the capitalized alphabet only, without adding any extra phrase or period.
ARC-C	Which is true? Output A, B, C, or D.
PubHealth (baseline)	Is the following statement correct or not? Say true if it's correct; otherwise, say false. Don't capitalize or add periods, just say "true" or "false".
PubHealth	Is statement 'true' or 'false'? Only output 'true' or 'false'.
ASQA (baseline)	Instruction: Write an ACCURATE, ENGAGING, and CONCISE answer for the given question using the retrieved graph information (some of which might be irrelevant). Use an unbiased and journalistic tone.
ASQA	Answer the following question. The question may be ambiguous and have multiple correct answers, and in that case, you have to provide a long-form answer including all correct answers. [Long Form]
ELI5 (baseline)	Instruction: Write an ACCURATE, ENGAGING, and CONCISE answer for the given question using the retrieved graph information (some of which might be irrelevant). Use an unbiased and journalistic tone.
ELI5	Provide a paragraph-length response using simple words to answer the following question. [Long Form]

Table 4. Task-specific instructions. For short-form QA, we do not use instructions and use the original questions only.

⁸All the Wikipedia dumps (2018, 2020) we used are downloaded from <https://github.com/facebookresearch/atlas>

F. Qualitative Examples

In this section, we show some qualitative examples and running examples of RAS.

Example 1 (PopQA)

[SUBQ] Who was the screenwriter for The Fake?

Retrieved Graph Information:

(S> The Fake | P> Release year | O> 1953),
 (S> The Fake | P> Genre | O> British crime film),
 (S> The Fake | P> Director | O> Godfrey Grayson),
 (S> The Fake | P> Stars | O> Dennis O'Keefe),
 (S> The Fake | P> Stars | O> Coleen Gray),
 (S> The Fake | P> Stars | O> Hugh Williams),
 (S> The Fake | P> Plot concerns | O> American detective solving theft of priceless painting),
 (S> Priceless painting | P> Stolen from | O> Tate Gallery),
 (S> Tate Gallery | P> Located in | O> London)

[SUBQ] Who wrote the screenplay for the 1953 British crime film The Fake?

Retrieved Graph Information:

(S> Antony Carr | P> Occupation | O> Author),
 (S> Antony Carr | P> Genre | O> Crime novels),
 (S> Antony Carr | P> Birth year | O> 1916),
 (S> Antony Carr | P> Death year | O> 1995),
 (S> Antony Carr | P> Nationality | O> English),
 (S> Antony Carr | P> Father | O> Robert Carr),
 (S> Robert Carr | P> Occupation | O> Singer),
 (S> Antony Carr | P> Niece | O> Susannah Carr),
 (S> Susannah Carr | P> Occupation | O> Newsreader),
 (S> Susannah Carr | P> Nationality | O> Australian)

[SUBQ] Who wrote the script or screenplay for the [1953 British crime film The Fake directed by Godfrey Grayson](#)?

Retrieved Graph Information:

(S> The Fake | P> Is a | O> British silent drama film),
 (S> The Fake | P> Release year | O> 1927),
 (S> The Fake | P> Directed by | O> Georg Jacoby),
 (S> The Fake | P> Stars | O> Henry Edwards),
 (S> The Fake | P> Stars | O> Elga Brink),
 (S> The Fake | P> Stars | O> Juliette Compton),
 (S> The Fake | P> Based on | O> Play of the same title),
 (S> Play of the same title | P> Written by | O> Frederick Lonsdale),
 (S> Play of the same title | P> Release year | O> 1924),
 (S> The Fake | P> Made at | O> Twickenham Studios),
 (S> Twickenham Studios | P> Located in | O> London)

Question: Who was the screenwriter for The Fake?

Answer: Frederick Lonsdale

Figure 23. An inference example in PopQA dataset.

Example 2 (TriviaQA)**[SUBQ]** Who is the most successful UK solo artist in the USA?**Retrieved Graph Information:**

(S> Jay Sean | P> Achievement | O> Most successful male UK urban artist in US chart history),
 (S> Jay Sean | P> Song reached number one | O> Down),
 (S> Jay Sean | P> Chart topped | O> Billboard Hot 100),
 (S> Taio Cruz | P> Topped chart | O> US Billboard Hot 100),
 (S> Cliff Richard | P> Claimed worldwide sales | O> 260 million),
 (S> Cliff Richard | P> Ranking | O> Second highest selling British male solo artist worldwide),
 (S> Elton John | P> Ranking | O> Highest selling British male solo artist worldwide),
 (S> Cliff Richard | P> Backing band | O> The Shadows),
 (S> Robbie Williams | P> Chart topper | O> Rock DJ),
 (S> Robbie Williams | P> Collaboration with | O> Kylie Minogue)

[SUBQ] What are the US chart records or album sales figures for Elton John in the United States?**Retrieved Graph Information:**

(S> Elton John | P> Plays | O> Piano),
 (S> Elton John | P> Performs | O> Vocals),
 (S> Dee Murray | P> Plays | O> Bass),
 (S> Nigel Olsson | P> Plays | O> Drums),
 (S> Davey Johnstone | P> Plays | O> Electric guitar),
 (S> Paul Buckmaster | P> Arranges | O> Orchestral arrangement),
 (S> Bob Birch | P> Plays | O> Bass guitar),
 (S> Charlie Morgan | P> Plays | O> Drums),
 (S> Charlie Morgan | P> Plays | O> Percussion),
 (S> Guy Babylon | P> Plays | O> Keyboards),
 (S> Matt Springer | P> Ranked | O> Elton John's studio albums),
 (S> Ultimate Classic Rock | P> Employs | O> Matt Springer),
 (S> Daryl Hall & John Oates | P> Nationality | O> American),
 (S> Daryl Hall & John Oates | P> Sold albums in US | O> 13 million),
 (S> Daryl Hall & John Oates | P> Sold singles in US | O> 6 million)

Question: Who is the most successful UK solo artist in the USA?**Answer:** Elton John

Figure 24. An inference example in TriviaQA dataset.

Example 3 (ASQA)

[SUBQ] Who sang the song i'm coming out?

Retrieved Graph Information:

(S> Diana Ross | P> Sang | O> I'm Coming Out),
 (S> Bernard Edwards | P> Wrote | O> I'm Coming Out),
 (S> Nile Rodgers | P> Wrote | O> I'm Coming Out),
 (S> Bernard Edwards | P> Produced | O> I'm Coming Out),
 (S> Nile Rodgers | P> Produced | O> I'm Coming Out),
 (S> I'm Coming Out | P> Released on | O> August 22, 1980),
 (S> I'm Coming Out | P> Second single from album | O> Diana),
 (S> Diana Ross | P> Commissioned | O> Chic),
 (S> Stevie J | P> Sampled | O> I'm Coming Out),
 (S> The Notorious B.I.G. | P> Used sample in song | O> Mo Money Mo Problems),
 (S> Marcia Hines | P> Covered | O> I'm Coming Out),
 (S> Keyshia Cole | P> Remade | O> I'm Coming Out),
 (S> Iggy Azalea | P> Featured on remake of | O> I'm Coming Out),
 (S> Ariana Grande | P> Sampled in song | O> Break Your Heart Right Back),
 (S> Hella von Sinnen | P> Sampled | O> I'm Coming Out),
 (S> Amerie | P> Covered | O> I'm Coming Out)

[Long Form] Question: Who sang the song I'm coming out?

Ground Truth: "I'm Coming Out" is a song recorded by American singer Diana Ross. It was written and produced by Chic members Bernard Edwards and Nile Rodgers, and released on August 22, 1980, as the second single from Ross' self-titled tenth album Diana (1980). In 2003, American singer Amerie covered "I'm Coming Out" for the soundtrack to the film "Maid in Manhattan". Her version was released as a single in select European countries and Australia.

Generation: "I'm Coming Out" is a song by American singer Diana Ross. It was written by Bernard Edwards and Nile Rodgers and produced by Edwards and Rodgers. The song was released on August 22, 1980 as the second single from Ross's album Diana. The song was sampled by Stevie J in his 2005 song "U Don't Know" and by The Notorious B.I.G. in his 1997 song "Mo Money Mo Problems". Marcia Hines covered the song for her 1981 album Marcia. Keyshia Cole remade the song for her 2007 album Just Like You. In 2014, Iggy Azalea featured on a remake of the song. Ariana Grande sampled the song in her 2015 song "Break Your Heart Right Back".

Example 4 (ELI5)

[SUBQ] What is a stock 'put' and how does it work?

Retrieved Graph Information:

(S> Put option | P> Is | O> Financial contract),
 (S> Put option | P> Gives buyer | O> Right to sell underlying asset),
 (S> Put option | P> Has | O> Predetermined price),
 (S> Put option | P> Has | O> Expiration date),
 (S> Put option | P> Can be associated with | O> Stocks),
 (S> Put option | P> Can be associated with | O> Commodities),
 (S> Put option buyer | P> Not obligated to | O> Sell at contract price),
 (S> Put option | P> Used for | O> Hedging),
 (S> Put option buyer | P> Bets on | O> Stock price decline),
 (S> Put option | P> Has | O> Defined risk)

[Long Form] Question: What is a stock 'put' and how does it work?

Ground Truth: A stock put is a type of stock option, that grants you the option to sell at a specified date for a set price. It's effectively a means of protecting stock you own, or for betting against a stock. Let's say you buy an 1 contract for August 140 puts on Apple. That means that the date August options expire, you would be able to sell 100 shares at \$140 if you so choose. So if Apple drops to \$125 by that time, you'd protect \$15/share of gains. This might only cost you \$1-2 per share to buy the contract. Or if you simply think the stock will fall, the put will increase in value as the strike price approaches "in the money." So that \$1-2/sh. you spend to buy the put option might be worth \$10/share if the stock starts dropping and there's greater likelihood that it'll be below the \$140 price. Because there is a set expiration date, the values have a relationship to both the underlying stock and the time until the expiration. So a month out, the put option that's \$10 in the money (\$140 put, stock trading at \$130) might cost \$20 while the day of the expiration it'd be just about the difference between option value and stock itself.

Generation: A put option is a financial contract that gives the buyer the right to sell the underlying asset at a predetermined price and expiration date. The put option buyer is not obligated to sell the asset at the contract price, but they can bet on the stock price decline.

Figure 25. Examples of long-form generation on ASQA and ELI5.

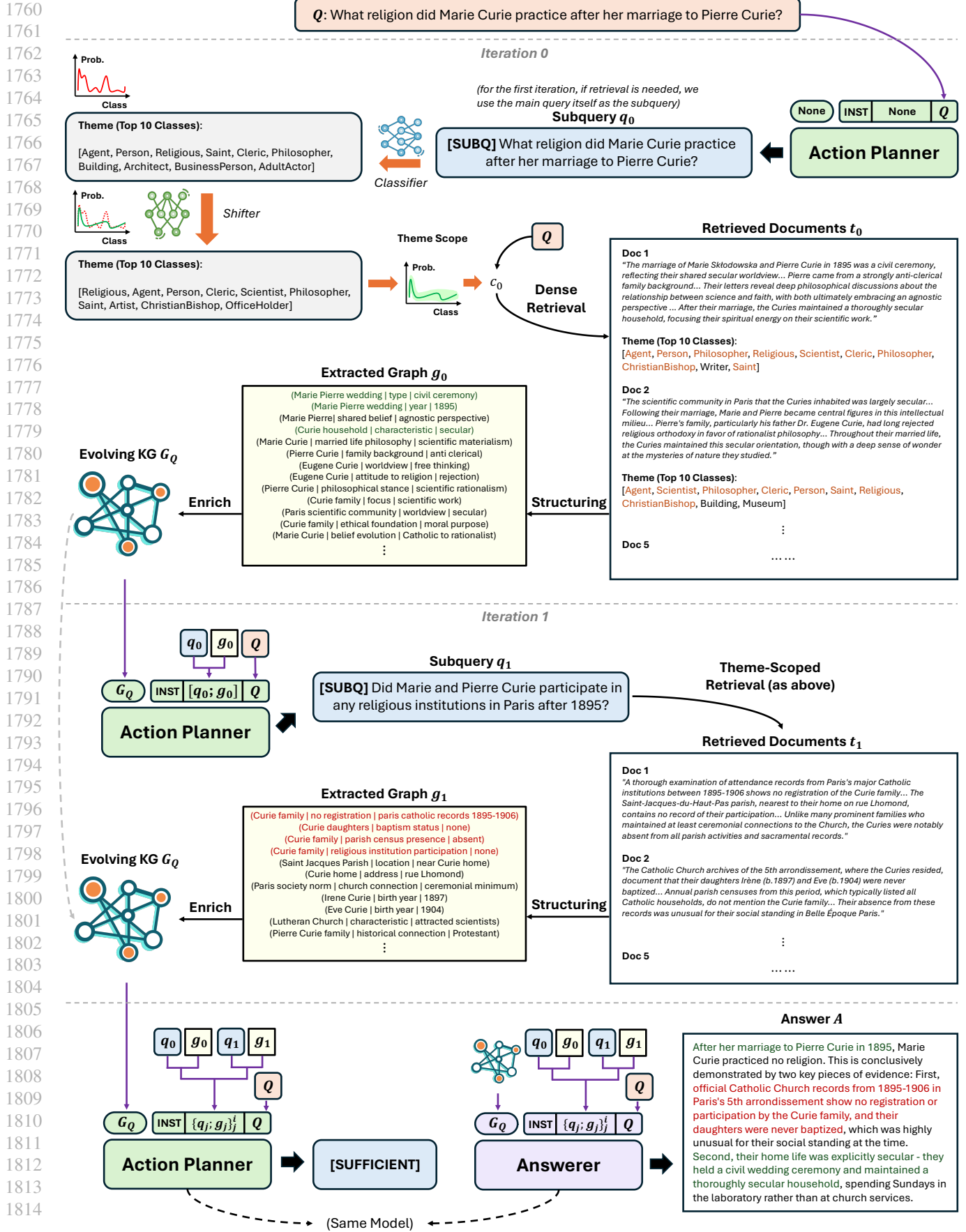


Figure 26. Running Example of RAS for Demonstration.

G. Hyperparameter Study

Table 5. Summary of hyper-parameter study for the experimental setup. We **highlight** the setting used in experiments.

Hyper-parameter	Studied Values
Theme Classifier	
Encoder learning rate	{1e-6, 1e-5, 2e-5 , 1e-4, 2e-4}
Classifier learning rate	{1e-6, 1e-5, 2e-5, 1e-4, 2e-4 , 5e-4, 1e-3}
Batch size	{4, 8, 16 , 32, 64}
Hidden layers	{ 1 , 2, 3}
Hidden dim	{256, 512 , 1024}
Dropout rate	{ 0.1 , 0.2, 0.3}
Distribution Shifter	
Batch size	{4, 8, 16, 32 , 64}
Learning rate	{0.0001, 0.0005, 0.001 , 0.005}
Hidden layers	{[256], [512], [512 , 256], [1024, 512, 256]}
Dropout rate	{0.1, 0.15, 0.2 , 0.25, 0.3}
Optimizer	{ Adam , SGD, RMSprop, AdamW}
Loss function	{ KLDiv , JSD, MSE, Wasserstein, CrossEntropy}
Text-to-Triples Model	
Batch size	{2, 4, 8 , 16, 32}
Learning rate	{1e-5, 2e-5, 5e-5 , 1e-4, 2e-4}
GraphLLM (Action Planner & Answerer)	
<i>GNN Setting (for Graph Token)</i>	
GNN architecture	{GCN, GAT, Graph Transformer }
Hidden dimension	{512, 768, 1024 , 2048}
Number of layers	{2, 3 , 4, 5}
Number of heads	{4, 6, 8 , 12}
Dropout rate	{0.05, 0.1 , 0.2, 0.3}
Projector intermediate dimension	{1024, 2048 , 4096}
Projector output dimension	4096
<i>LoRA Setting</i>	
LoRA rank (r)	{4, 8 , 16, 32}
LoRA alpha	{8, 16 , 32}
LoRA dropout	{0.01, 0.05 , 0.1, 0.15}
<i>General Setting</i>	
Learning rate	{1e-6, 2e-6, 5e-5 , 1e-4}
Batch size (training)	{ 2 , 4, 8, 16, 32}
Batch size (inference)	1
Weight decay	{0.001, 0.01 , 0.05, 0.1}
Gradient accumulation steps	{ 2 , 4, 8, 16}
Gradient clipping	{0.1, 0.3, 0.5 , 1.0}
Warmup ratio	{0.05, 0.1, 0.15 , 0.2}
Max text length	2500
Max new tokens	Task-specific (see Appendix E.2)
Others	
Theme scoping top- K	{100,000, 250,000 , 500,000 }
Dense retrieval top- k	5

H. Notation Table

Table 6. Notation used throughout the paper

Notation	Description
<i>General</i>	
Q	Main input query
A	Generated answer
C	Complete document corpus
C_{PPD}	Theme-indexed document corpus
<i>Action Planning</i>	
$\mathcal{M}_{plan}$	Action planner model
p_i	Plan generated at iteration i
q_i	Subquery generated at iteration i
g_i	Graph information (triple list) extracted at iteration i
<i>Theme-Scoped Retrieval</i>	
f_{cls}	Theme classifier function
f_{shift}	Theme distribution shifter function
f_{den}	Dense retriever function
$PPD(q_i)$	Posteriori probability distribution of query q_i
$PPD(d_i)$	Expected document theme distribution
c_i	Theme-specific sub-corpus at iteration i
t_i	Retrieved text passages at iteration i
K	Number of documents selected in theme scoping
k	Number of passages selected in dense retrieval
<i>Knowledge Structuring</i>	
f_{t2t}	Text-to-triple conversion model
f_{enc}	Text encoder for node/edge attributes
f_{gnn}	Graph neural network encoder
G_i	Knowledge graph at iteration i
G_Q	Final query-specific knowledge graph
V_i, E_i	Node and edge sets of graph at iteration i
V_Q, E_Q	Node and edge sets of final graph
$v.attr$	Attribute vector of node v
$e.attr$	Attribute vector of edge e
<i>Answer Generation</i>	
$\mathcal{M}_{ans}$	Answerer model
$\hat{A}$	Model-generated answer before retrieval